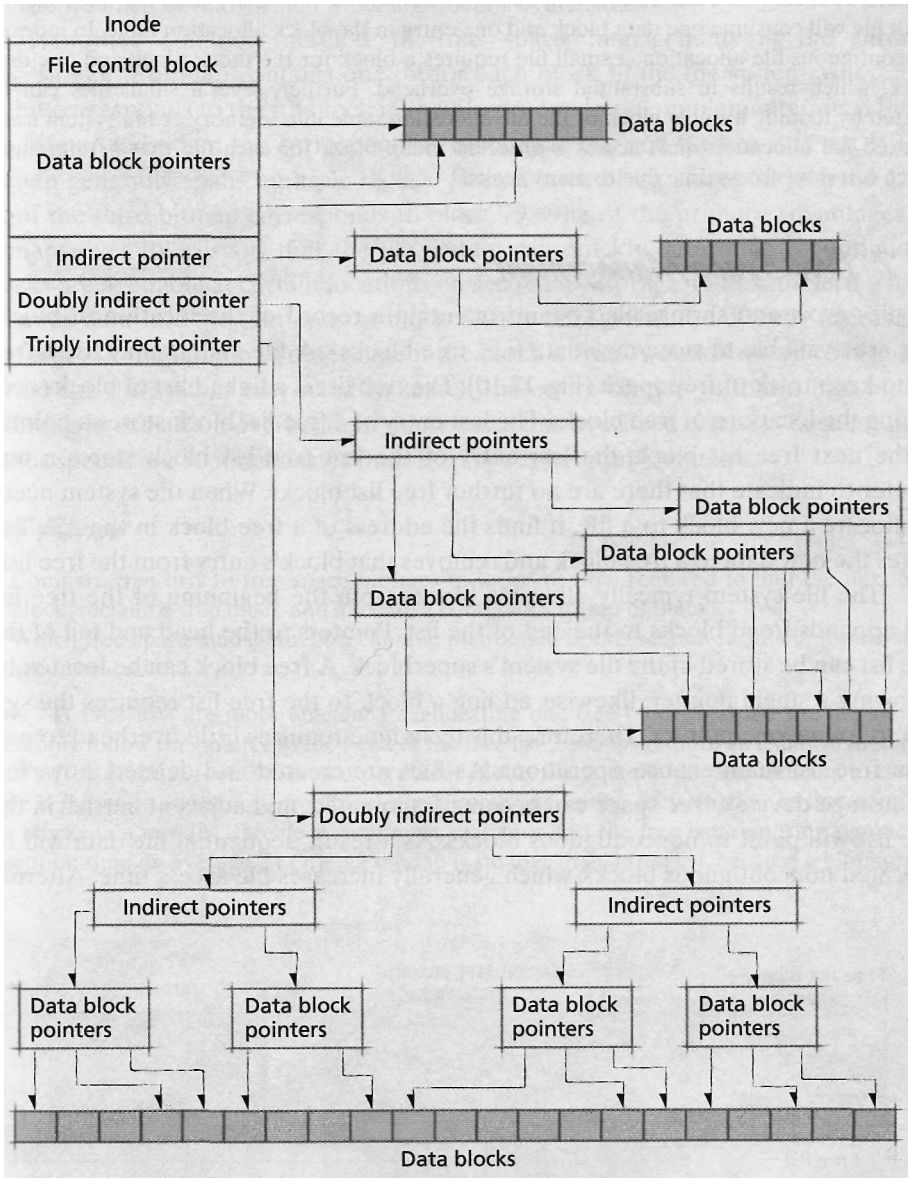


Figure 13.9 | *Inode structure.*

Self Review

1. How does placing index blocks near the data they reference improve access time?
2. Compare indexed noncontiguous file allocation to tabular noncontiguous file allocation when files are, on average, small (e.g., less than or equal to one block).

Ans: **1)** When the file system reads an index block, the disk arm will be near the data it references, reducing or even eliminating seeks. **2)** In tabular noncontiguous file allocation, a small file will consume one data block and one entry in the block allocation table. In indexed noncontiguous file allocation, a small file requires a block for the index block and one data block, which results in substantial storage overhead. Further, several small files can be located by loading a single block of the file allocation table into memory. A file system usinsg indexed file allocation must access a different index block for each file that it references, which can slow access time due to many seeks.

13.7 Free Space Management

As files grow and shrink, file systems maintain a record of the location of blocks that are available to store new data (i.e., free blocks). A file system may use a **free-list** to keep track of free space (Fig. 13.10). The free list is a linked list of blocks containing the locations of free blocks. The last entry of a free list block stores a pointer to the next free list block; the last entry of the last free list block stores a null pointer to indicate that there are no further free list blocks. When the system needs to allocate a new block to a file, it finds the address of a free block in the free list. writes the new data to a free block and removes that block's entry from the free list.

The file system typically allocates blocks from the beginning of the free list and appends freed blocks to the end of the list. Pointers to the head and tail of the free list can be stored in the file system's superblock. A free block can be located by following a single pointer; likewise, adding a block to the free list requires the system to follow one pointer. Therefore, this technique requires little overhead to perform free list maintenance operations. As files are created and deleted, however, the storage device's free space can become fragmented, and adjacent entries in the free list will point to noncontiguous blocks. As a result, sequential file data will be allocated noncontiguous blocks, which generally increases file access time. Alterna-

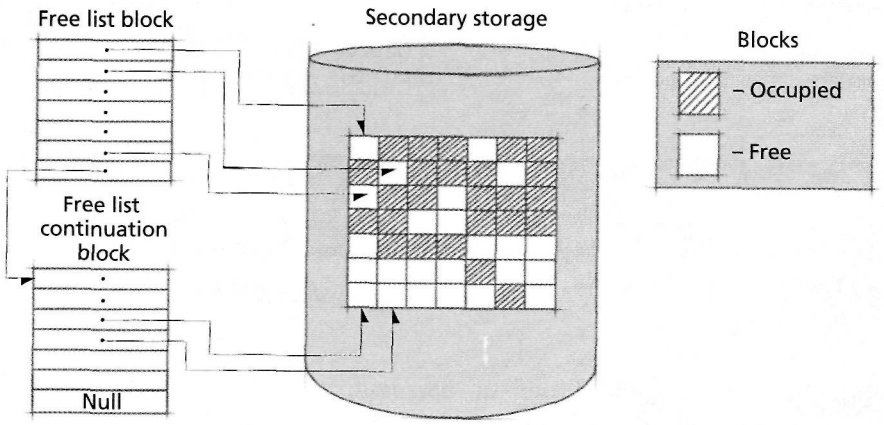


Figure 13.10 | *Free space management using a free list.*

tively, the file system can attempt to allocate contiguous blocks by searching or sorting the free list, both of which incur significant overhead.[48]

Another common method of free space management is the **bitmap** (Fig. 13.11). A bitmap contains one bit for each block in the file system, where the *i*th bit corresponds to the *i*th block in the file system. In one implementation, a bit in the bitmap is 1 when the corresponding block is in use and 0 when it is not.[49] The bitmap generally spans multiple blocks. Thus, if each block stores 32 bits, the 15th bit of the third bitmap corresponds to block 79. One of the primary advantages to bitmaps over free lists is that the file system can quickly determine if contiguous blocks are available at certain locations on secondary storage. For example, if a user appends data to a file that ends at block 60, the file system can directly access the 61st entry of the bitmap to determine if the block is free. A disadvantage to bitmaps is that the file system may need to search the entire bitmap to find a free block, resulting in execution overhead. In many cases this overhead is trivial, because processor speeds are so much faster than I/O speeds in today's systems.

Self Review

1. Compare free lists to free space bitmaps in terms of time required to find the next free block, reclaim a free block and allocate a contiguous group of blocks.
2. Which free space management technique mentioned in this section results in lowest storage overhead?

Ans: **1)** Free lists are more efficient for allocating one free block, because the file system need only follow the pointer to the head of the free list. Bitmaps require an exhaustive search until a free block is found. Given a particular block, file systems can use bitmaps to determine if there are contiguous free blocks by directly inspecting their entries. To find contiguous blocks in a free list, the file system must search or sort the free list, requiring significant execution time on average. **2)** Often a bitmap is smaller than a free list, because a bitmaprep-

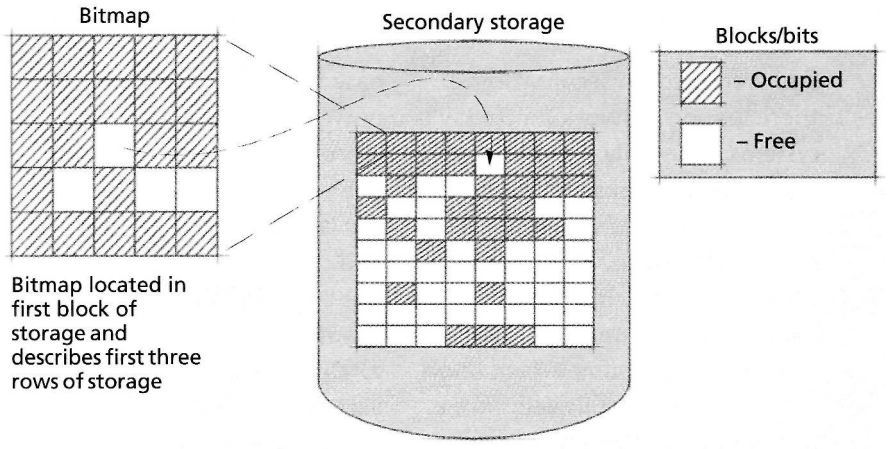


Figure 13.11 | *Free space management using a bitmap.*

resents each block using a single bit, but a free list uses a block number, which can be as large as 32 or 64 bits. A file system's bitmap size is constant, while the free list's size depends on the number of free blocks in the system. Thus, when a file system contains few free blocks, a free list consumes less space than a bitmap.

13.8 File Access Control

Files are often used to store sensitive data such as credit card numbers, passwords, social security numbers and more, so file systems should include mechanisms to control user access to data (see the Operating Systems Thinking feature, Security). In the sections that follow, we discuss common techniques for implementing file access control.

13.8.1 Access Control Matrix

One way to control access to tiles is to create a two-dimensional **access control matrix** (Fig. 13.12) listing all the users and all the files in the system. The entry a_{ij} is 1 if user *i* is allowed access to file *j*; otherwise a_{ij} - 0. For example, in Fig. 13.12, user 5 can access all ten files and user 4 can access only file 1. In an installation with a large number of users and a large number of files, this matrix would be quite large. Further, allowing one user access to another user's files is the exception rather than the rule, so the matrix would be extremely sparse. To make such a matrix concept useful, it would be necessary to use codes to indicate various kinds of access such as read-only, write-only, execute-only, read/write, and so on. This could substantially increase the size of the matrix.

Operating Systems Thinking

Security

Computer security has always been an important concern, especially for people responsible for business-critical and mission-critical systems. But the nature of the security problems changes with advances in technology, so operating systems designers must always be evaluating new technology trends and their susceptibility to attack. Early computer systems often weighed many tons and were kept in secure rooms; there was no networking. Today's systems, especially hand-held computers, laptops and even desktop machines can easily be stolen. Now, virtually all systems are capable of being networked, which creates numerous challenges as proprietary information is transmitted over insecure transmission media. Early encryption/decryption schemes have become easy to "crack" with today's high-powered computers, so much more sophisticated schemes have been developed. We discuss security issues throughout the book and devote all of Chapter 19 to this crucial topic. The case study chapters on Linux and Windows XP discuss security considerations and capabilities in those popular operating systems.

User \ File	1	2	3	4	5	6	7	8	9	10
1	1	1	0	0	0	0	0	0	0	0
2	0	0	1	0	1	0	0	0	0	0
3	0	1	0	1	0	1	0	0	0	0
4	1	0	0	0	0	0	0	0	0	0
5	1	1	1	1	1	1	1	1	1	1
6	0	0	0	0	0	1	1	0	0	0
7	1	0	0	0	0	0	0	0	0	1
8	1	0	0	0	0	0	0	0	0	0
9	1	1	1	1	0	0	0	0	1	1
10	1	1	0	0	1	1	0	0	0	1

Figure 13.12 *| Access control matrix.*

Self Review

1. Why is file access control necessary?
2. Why are access control matrices inappropriate for most systems?

Ans: **1)** Any user can reference any pathname in a file system. File systems control access to files to protect personal and sensitive information from users that do not have proper authorization. **2)** Access control matrices are generally large and sparsely populated, leading to wasted storage and inefficient access times when enforcing access control policies.

13.8.2 Access Control by User Classes

A technique that requires considerably less space than the uses of an access control matrix is the control of access to various **user classes.** A common file access classification scheme is

- **Owner**—Normally, this is the user who created the file. The owner has unrestricted access to the file and typically can change file permissions.
- **Specified user**—The owner specifies that another individual may use the file.
- **Group** (or project)—Users are often members of a group working on a particular project. In this case the various members of the group may all be granted access to each other's project-related files.
- **Public**—Most systems allow a file to be designated as public so that it may be accessed by any member of the system's user community. By default, public access rights typically allow users to read or execute a file, but not to write it.

This access control data can be stored as part of the file control block and often consumes an insignificant amount of space. Security is an important issue in operating system design and is covered in greater detail in Chapter 19, Security.

Self Review

1. How do user classes reduce the storage overhead incurred by access control information?
2. Describe the advantages and disadvantages of storing access control data as part of the file control block.

Ans: **1)** User classes enable the file owner to grant permissions to a group of users using one entry. **2)** The advantage is that there is essentially no overhead if the user is granted access to a file, because the file system needs to read the file control block before opening the file anyway. However, if access is denied, the file system will have wastefully performed several lengthy seeks to access the file control block for a file that the user cannot open.

13.9 Data Access Techniques

In many systems, several processes may be requesting file data from various files spread across the storage device, leading to many seeks. Instead of instantly responding to immediate user I/O demands, the operating system may use several techniques to improve performance.

Today's operating systems generally provide many access methods. These are sometimes grouped as **queued access methods** and **basic access methods.** The queued methods provide more powerful capabilities than the basic methods.

Queued access methods are used when the sequence in which records are to be processed can be anticipated, such as in sequential and indexed sequential accessing. The queued methods perform **anticipatory buffering** and scheduling of I/O operations. Such methods attempt to have the next record available for processing as soon as the previous record has been processed. More than one record at a time is maintained in main memory; this allows processing and I/O operations to be overlapped, improving performance.

The basic access methods are normally used when the sequence in which records are to be processed cannot be anticipated, particularly with direct accessing. Also there are many situations, such as database applications, in which user applications want to control record accesses without incurring the overhead of anticipatory buffering. In the basic methods, the access method reads and writes physical blocks; blocking and deblocking (if appropriate to the application) is performed by the user application.

Memory-mapped files map file data to a process's virtual address space instead of using a file system cache.[50] Because references to memory-mapped files occur in a process's virtual address space, the virtual memory manager can make page-replacement decisions based on each process's reference pattern. Memory-mapped files also simplify application programming, because developers can access file data using pointers instead of specifying read, write and seek operations.

When a process issues a write request, the data is typically buffered in main memory (to improve I/O performance) and its corresponding page is marked as dirty. When a memory-mapped file's modified page is replaced, the page is written to its corresponding file on secondary storage. When the file is closed, the system flushes all dirty pages to secondary storage. To reduce the risk of data loss due to a system failure, dirty memory-mapped pages are flushed typically to secondary storage periodically.[51]

Self Review

1. Compare and contrast queued access methods and basic access methods.
2. How do memory-mapped files simplify application programming?

Ans: **1)** Queued access methods perform anticipatory buffering, which attempts to load into memory a block that is likely to be used in the near future. Basic access methods do not attempt to schedule or buffer I/O operations, which is more appropriate when the system cannot predict future requests or when the file system must reduce the risk of losing data during a system failure. **2)** Memory-mapped files enable programmers to access file data using pointers instead of file operations such as read, write and seek.

13.10 Data Integrity Protection

Computer systems often store critical information, such as inventories, financial records and/or personal information. System crashes, natural disasters and malicious programs can destroy this information. The results of such events can be catastrophic.[52] Operating systems and data storage systems should be fault tolerant in that they account for the possibility of disasters and provide techniques to recover from them.

13.10.1 Backup and Recovery

Most systems implement backup techniques to store redundant copies of information and recovery techniques that enable the system to restore data after a system failure. Backup and recovery strategies can also protect systems against user-generated events, such as an unintentional deletion of important data (see the Operating Systems Thinking feature, Backup and Recovery).

Physical safeguards are the lowest level of data protection. Physical obstacles such as locks and alarm systems can prevent unauthorized access to computers that store sensitive data. Because main memory typically is volatile (i.e., it loses its contents when the power is shut off), power outages can result in the loss of data not yet transferred to secondary storage. An uninterruptable power supply (UPS) can be used to protect data from being lost due to a power outage.[53]

Natural disasters such as fires and earthquakes can destroy all data at a site. Thus, some organizations maintain a backup site, geographically removed from the primary site to protect data in case of a site failure.[54] Although these precautions

are important, they do not protect data from operating system crashes or disk-head malfunctions.

Performing periodic backups is the most common technique used to prevent data loss. **Physical backups** duplicate a storage device's data at the bit level. In some cases, the system copies only allocated blocks of data. Physical backups are simple to implement, but they store no information about the logical structure of the file system. The file system's data may be stored in different formats, depending on the system architecture, so physical backups cannot easily be restored on computers using different architectures. Also, because a physical backup does not read the file system's logical structure, it cannot distinguish among the files it contains. Thus, physical backups must record and restore the entire file system to ensure that all data has been duplicated, even if most of a file system's data has not been modified since the last physical backup.[55]

A **logical backup** stores file system data and its logical structure. Thus, logical backups inspect the directory structure to determine which files need to be backed up, then write these files to a backup device (e.g., a tape, CD or DVD) in a common, often compressed, archival format. For example, the tape archive (tar) format is commonly used to store, transport and back up multiple files on UNIX-based systems (see www.gnu.org/software/tar/tar.html).Because logical backups store data in a common format using a directory structure, they permit operating systems with different native file formats to read and restore the backup data, so file data can be restored on multiple heterogeneous systems. Logical backups also enable the user to restore a single file from the backup (e.g., an accidentally deleted file),

Operating Systems Thinking

Backup and Recovery

Have you ever lost several hours work because of a system failure? HMD was working on a massive project with hundreds of software engineers when lightning struck our building and we lost all of the system updates for the last week. Fortunately, each of us had backups of our own work, but it still took us several days to recover—a painful and costly experience.

Should users be responsible for performing (potentially time-consuming) backups or should these be done automatically by the system? Backups consume significant resources. How often should backups be done? How much data should be backed up each time? Should backups be done as full system bit images or should they be done with incremental logging of each change to the system? Answering these questions requires the system administrator to balance the overhead of performing frequent backups with the risk of lost work due to infrequent backups. Incorporating backup and recovery features into operating systems is crucial.

which is typically faster than restoring the entire file system. However, because logical backups can read only data exposed by the file system, they may omit information such as hidden files and metadata that are copied by a physical backup when copying each bit on the file system's storage device. Saving files in a common format can be inefficient due to the overhead incurred when translating between the native file format and the archival format.[56]

Incremental backups are logical backups that store only file system data that has changed since the previous backup. The system can record which files have been modified and write them to the backup file periodically. Because incremental backups require less time and fewer resources than backing up an entire file system, they can be performed more often, reducing the risk of lost data due to disasters.

Self Review

1. Compare and contrast logical backups and physical backups.
2. Why are physical safeguards insufficient for preventing loss of data in the event of a disaster?

Ans: **1)** Logical backups are readable by file systems with different file formats, support incremental backups and enable fine-grained recovery, such as restoring a single file. Physical backups are typically easier to create, because the file system structure does not need to be traversed. However, physical backups do not support incremental or partial backups and are typically incompatible with different systems. **2)** Physical safeguards prevent access to data but do not prevent loss of data due to natural disasters, such as fires and earthquakes, or to hardware and power failures. In fact, there is no way to guarantee absolute security of files.

13.10.2 Data Integrity and Log-Structured File Systems

None of the techniques discussed so far address the possibility that significant activity may occur between the time of the last backup and the time at which a failure occurs (see the Operating Systems Thinking feature, Murphy's Law and Robust Systems). Further, backups often require a lengthy restoration process, during which time the system is not operational.

In systems that cannot tolerate data loss or downtime, RAID and transaction-based file systems are appropriate. In Section 12.10, Redundant Arrays of Independent Disks (RAID), we discussed how RAID levels 1-5 improve a system's mean-time-to-failure and can restore data in the event of a single disk failure. We also discussed how mirroring and hot swappable disks enable RAID systems to continue to operate when disks fail, providing high availability.

Logging and Shadow Paging

If a system failure occurs during a write operation, the file system data may be left in an inconsistent state. For example, an electronic transfer of funds might require a banking system to withdraw money from one account and deposit it in another. If the system fails between withdrawing and depositing the funds, it could lose the money. Transaction-based logging reduces the risk of data loss by using atomic **transactions**, which perform a group of operations in their entirety, or not at all. If

an error occurs that prevents a transaction from completing, it is **rolled back by** returning the system to the state before the transaction began.[57]

Atomic transactions can be implemented by recording the result of each operation in a log file instead of modifying existing data. Once the transaction has completed, it is **committed** by recording a special value in the log. At some time in the future, the log is transferred to permanent storage. If the system fails before the transaction completes, any operations recorded after the previous committed transaction are ignored. When the system recovers, it reads the log file and compares it to the data in the file system to determine the state of the file system at the last conmmit point.

To enable the system to undo any number of operations, in many systems the log file is not deleted after its operations are written to permanent storage. Because logs can become large, reprocessing transactions to find the state of the system at the last commit point can be time consuming. To reduce the time spent reprocessing transactions in the log, most transaction-based systems maintain **checkpoints** that point to the last transaction that has been transferred to permanent storage. If the system crashes, it need only examine transactions after the checkpoint.

Shadow paging implements atomic transactions by writing modified data to a free block instead of the original block. Once the transaction commits, the file system updates its metadata to point to the new block and releases the old block, or **shadow page,** as free space. If the transaction fails, the file system rolls back the transaction by releasing the new blocks as free space.[58]

Operating Systems Thinking

Murphy's Law and Robust Systems

Most everyone is familiar with one form or another of Murphy's Law, named for U.S. Air Force Capt. Edward Murphy who cursed one of his error-prone technicians by saying, "If there is any way to do it wrong, he'll find it." Today's most common variant is "If something can go wrong, it will." Often, "and at the most inopportune time" is appended to the law. Operating system designers are well advised to keep Murphy's Law in mind. They need to constantly ask questions like, "What can go wrong?" "What is the likelihood of such problems?" "What are the consequences of such problems?" "How can the operating system be designed to prevent such problems?" "If certain problems cannot be prevented, how should the operating system deal with them?" A robust system deals with a wide range of inputs and unexpected situations in a manner that allows the system to keep operating.

Log-Structured File Systems (LFS)

Transaction logging and shadow paging prevent file system data from entering an inconsistent state but do not necessarily guarantee that the file system itself will be in a consistent state. For example, moving a file from one directory to another requires the file system to delete the file's original directory entry and create an entry in its new directory. A system failure (e.g., due to a power failure) that occurs between deleting the original directory entry and creating the new one can result in loss of file data. To address this limitation, a **log-structured file** system **(LFS),** also called a **journaling file system,** performs file system operations as logged transactions.[59] Examples of log-structured file systems include Microsoft's NTFS Journaling File System and the Red Hat's ext3 file system for Linux.[60]-[61]

In an LFS, the entire disk serves as a log file to record transactions. New data is written sequentially in the log file's free space. For example, in Fig. 13.13, the LFS receives a request to create files f oo and bar in a new directory. The file system performs the requested operation first by writing f oo's data to the log, then by writing foo's metadata (e.g., an inode), which enables the file system to locate foo's data. Similarly, the LFS writes bar's data and corresponding file metadata. Finally, the file system writes the new directory entry to the log. Note that if the file system writes a file's metadata before writing file data to the log, the system could fail before the file's data is written. This could leave the file system in an inconsistent state, because metadata will reference invalid data blocks (i.e., those that were not written before the system failed).

Because modified directories and metadata are always written to the end of the log, an LFS might need to read the entire log to locate a particular file, leading to poor read performance. To reduce this problem, an LFS caches locations of file system metadata and occasionally writes **inode maps** or superblocks, which indicate the location of other metadata (Fig. 13.13). This enables the operating system to locate and cache file metadata quickly when the system boots. Subsequently, file data can be accessed quickly by determining its location from the file system's caches. As the size of the file system cache increases, its performance improves at the cost of reducing the amount of memory available to user processes.[62]

To reduce overhead, some file systems store only metadata in the log. In this case, the file system modifies metadata first by writing an entry to the log, then by updating the entry in the file system. The operation commits only after the metadata has been updated both in the log and in the file system. This ensures file system integrity with relatively low overhead but does not ensure file integrity in the event of a system failure.[63]

Because data is written sequentially in an LFS, each write request is performed sequentially, which can substantially reduce write times. By comparison, noncontiguous file allocation implementations might require the file system to traverse the directory structure, which may be distributed throughout the disk, leading to long access times.

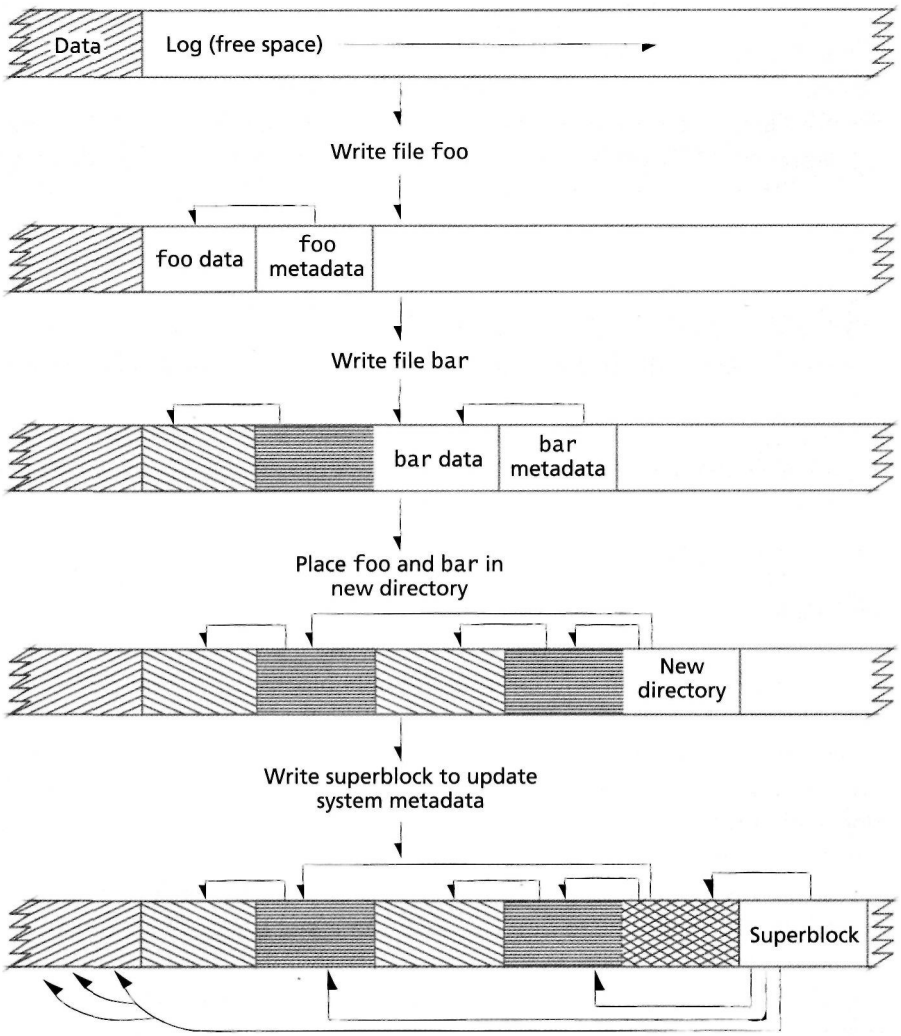


Figure 13.13 | *Log-structured file system.*

When the log fills, an LFS must determine how to recover free space for incoming data. An LFS can inspect the content of the log periodically to determine which blocks can be freed because the log contains a modified copy of their data. The LFS can thread new data through these blocks, which are likely to be highly fragmented. Unfortunately, threading can reduce read and write performance to levels lower than those in conventional file systems, because the disk might need to perform many seeks to access data. To address this issue, an LFS can create contiguous free space in the log by copying data to a contiguous region at the end of the log. While performing this I/O-intensive operation, however, users will experience poor response times.[64]

1. Explain how data stored by a RAID level 1 array, which offers a high level of redundancy, could enter an inconsistent state.
2. Explain the difference between logging and shadow paging.

Ans: **1)** If a power failure occurs while data in a mirrored pair is being written, the array may contain incomplete copies of file data or file system metadata on both disks. **2)** Logging keeps track of when a transaction was started, and when it completed. If the transaction fails, it rolls back to the last commit point. Shadow paging involves allocating new data blocks to write the new data, then freeing the old data blocks when the transaction is complete.

13.11 File Servers and Distributed Systems

One approach to handling nonlocal file references in a computer network is to route all such requests to a **file server,** i.e., a computer system dedicated to resolving intercomputer file references.[65, 66, 67, 68, 69] This approach centralizes control of these references, but the file server could become a bottleneck, because all client computers would send all requests to the server. A better approach is to let the separate computers communicate directly with one another; this is the approach taken by Sun Microsystem's **Network File System (NFS).** In an NFS network, each computer maintains a file system that can act as a server and/or a client.

Through the 1970s, file systems generally stored and managed the files of a single computer system. On multiuser timesharing systems the files of all users were under the control of a centralized file system. Today, the trend is toward **distributed** file systems in computer networks.[70, 71, 72, 73] A real complication is that such networks often connect a wide variety of computer systems with different operating systems and file systems.

A distributed file system enables users to perform operations on remote files in a computer network in much the same manner as on local files.[74, 75, 76, 77, 78] NFS provides distributed file system capabilities for networks of heterogeneous computer systems; it is so widely used that it has become an international standard.[79] File servers and distributed file systems are discussed in Section 18.2, Distributed File Systems.

Self Review

1. Why is a single file server impractical for environments such as large organizations?
2. What is the primary advantage to having each client in a distributed file system also be a server?

Ans: **1)** If the file server fails, the whole file system becomes unavailable. A central file server can also become a bottleneck. **2)** This removes the bottleneck of central file servers and can enable the file system to be operational even if one or more clients fail.

13.12 Database Systems

A database is a centrally controlled, integrated collection of data; a **database system** involves the data, the hardware on which the data resides and the software that

controls access to data (called a **database management system** or **DBMS).** Databases are commonly implemented on Web servers and in online transaction-processing environments, where multiple processes require quick access to a large store of shared data.

13.12.1 Advantages of Database Systems

In conventional file systems, multiple applications often store the same information at different pathnames and in different formats (e.g., a text file stored as a Postscript and as a PDF file). To eliminate this redundancy, database systems organize data according to its content instead of by pathname. For example, a Web site may contain separate applications to charge orders to customer credit cards, send bills to those customers and print labels for packages to be delivered to customers. All of these applications must access customer information such as name, address, phone number, and the like. However, each application must format the information differently to perform its task (e.g., send credit card information or print a label). A database would store only one copy of each customer's information and enable each application to access the information and format it as required. To enable this access to shared data, databases incorporate a querying mechanism that allows applications to specify which information to retrieve.

Database systems use standardized data organization techniques (e.g., hierarchical, relational, object oriented), the structure of which cannot be altered by applications. Database systems are susceptible to attack due to centralized control and location of data, so they are designed with elaborate security mechanisms.[80, 81]

Self Review

1. How do database systems reduce redundant data typically stored by traditional file systems?
2. Why is security extremely important in database systems?

Ans: **1)** Database systems address data according to its content so that no two records contain the same information. For example, a database might store a single copy of customer information and enable multiple applications to access that information using queries. **2)** Databases provide centralized control to large stores of data, which may contain sensitive information. If a malicious user gains unauthorized access to a database, that user may be able to access all data stored by the system. This is more serious than in traditional file systems, because access rights are typically stored for each file, so a user that obtains unauthorized access to a file system may not be able to access all of its files.

13.12.2 Data Access

Database systems exhibit **data independence,** because a database's organizational structure and access techniques are hidden from applications. Unlike a hierarchically structured file system, which requires all applications to access data using pathnames, a data-independent system enables multiple applications to access the same data using different logical views. Consider the customer information stored by the Web site in the preceding section. In this case, a credit card transaction pro-

cessing application may view customer information as a list of credit card number and expiration dates; the application that performs billing may view customer information as names, addresses and dollar amounts. Data independence makes it possible for the system to modify its storage structure and data access strategy in response to the installation's changing requirements without needing to modify functioning applications.

Database languages facilitate data independence by providing a standard way to access information. A database language consists of a **data definition language (DDL),** a **data manipulation language (DML)** and a **query language.** A **DDL** specifies how data items are organized and related and a DML is used to modify data. A query language allows users to create queries, which search the database for data that meets certain criteria. Database languages can be executed directly from a command line or through programs written using higher-level host languages, such as C++ or Java. The **Structured Query Language (SQL),** which consists of a DDL, DML and a query language, is a popular query language that enables users to find data items that have certain properties, create tables, specify integrity constraints, manage consistency and enforce security.[82, 83, 84]

A **distributed database** is a database that is spread throughout the computer systems of a network. Distributed databases facilitate efficient data access across many sets of data that reside on different computers.[85] In such systems each data item is typically stored at the location at which it is most frequently used, while remaining accessible to other network users. Distributed systems provide the control and efficiency of local processing with the advantages of information accessibility over a geographically dispersed organization. They can be costly to implement and operate, however, and they can suffer from increased vulnerability to security attacks and breaches.

Self Review

1. How do databases facilitate access to data for application developers?
2. Explain the difference between a DDL and a DML.

Ans: **1)** Database languages facilitate data independence, so programmers can access data according to a logical view that is most appropriate for their applications. **2)** A DDL defines the organization of and relationships between data elements. A DML is used to modify data.

13.12.3 Relational Database Model

Databases are based on models that describe how data and their relationships are viewed. The relational model developed by Codd is a logical structure rather than a physical one; the principles of relational database management can be considered without concerning oneself with the physical implementation of the underlying data structures.[86,87,88,89,90]

A relational database is composed of **relations** (tables). Figure 13.14 illustrates a relation that might be used in a personnel system. The name of the relation is EMPLOYEE and its primary purpose is to organize the various attributes of each

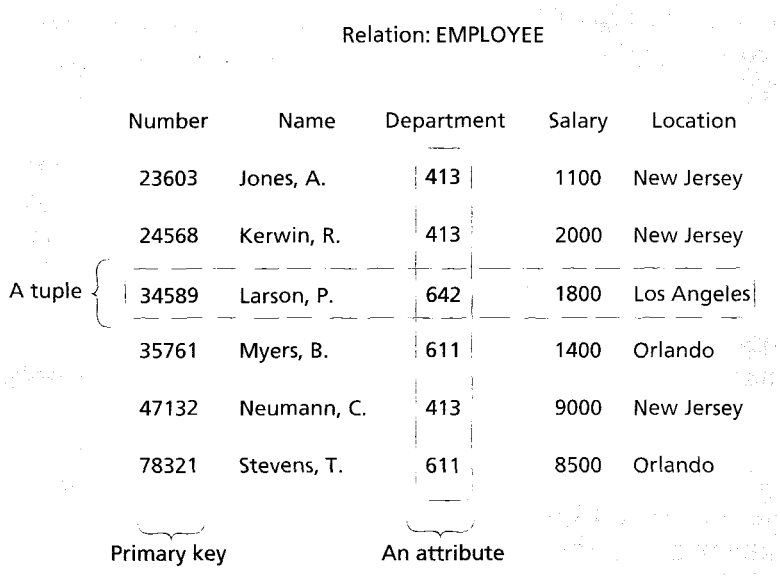


Figure 13.14 | *Relation in a relational database.*

employee. Any particular element (row) of the relation is called a **tuple.** This relation is a set of six tuples. In this example, the first attribute (column) in each tuple. the employee number, is used as the **primary key** for referencing data in the relation. The tuples of the relation are uniquely identifiable by primary key.

Each attribute of the relation belongs to a single **domain.** Tuples must be unique within a relation, but particular attribute values may be duplicated between tuples. For example, three different tuples in the example contain department number 413. The number of attributes in a relation is the **degree** of the relation. Relations of degree 2 are **binary relations,** relations of degree 3 are **ternary relations,** and relations of degree *n* are **n-ary relations.**

Users of a database are often interested in different data items and different relationships among them. Most users will want only certain subsets of the table rows and columns. Many users will wish to combine smaller tables into larger ones to produce more complex relations. Codd called the subset operation **projection** and the combination operation **join.**

Using the relation of Fig. 13.14 we might, for example, use the projection operation to create a new relation called DEPARTMENT-LOCATOR whose purpose is to show where departments are located (Fig. 13.15).

The Structured Query Language (SQL) operates on relational databases. The SQL query in Fig. 13.16 generates the table in Fig. 13.15. Line 1 begins with "--", the SQL delimiter for a comment. The SELECT clause (line 2) specifies the columns. Department and Location, of the new table.The keyword DISTINCT indicates that the table should contain only unique entries. The FROM clause (line 3) indicates that

Relation: ***DEPARTMENT-LOCATOR***

Department	*Location*
413	NEW JERSEY
611	ORLANDO
642	LOS ANGELES

Figure 13.15 | *Relation formed by projection.*

```
1  -- SQL query to generate the table in Fig, 13,15
2  SELECT DISTINCT Department, Location
3  FROM EMPLOYEE
4  ORDER BY Department ASC
5
```

Figure 13.16 | *SQL query.*

the columns will be projected from the EMPLOYEE table (Fig. 13.14). On line 4, an ORDER BY clause indicates the column (in this case, Department) that is used to sort the query result and ASC indicates that the sort will be in ascending order.

The relational model has several advantages.

1. The tabular representation used in implementations the relational model is easy to implement in a physical database system.
2. It is relatively easy to convert virtually any other type of database structure into the relational model. Thus the model may be viewed as a universal form of representation.
3. The projection and join operations are easy to implement and make the creation of new relations required for particular applications easy to perform.
4. Access control to sensitive data is straightforward to implement. The sensitive data is merely placed in separate relations, and access to these relations is controlled by some sort of authority or access scheme.

Self Review

1. What is the difference between the join and projection operations in relational databases?
2. (T/F) In a relation, each attribute value must be unique among tuples.

Ans: **1)** The join operation forms a relation from more than one relation; the projection operation forms a relation from a subset of a relation's attributes. **2)** False. Multiple tuples may have the same attribute value.

13.12.4 Operating Systems and Database Systems

Stonebraker discusses various operating system services that support database management systems, namely buffer pool management, the file system, scheduling, process management, interprocess communication, consistency control and paged virtual

memory.[91] He observes that because most of these features are not specifically optimized to DBMS environments, DBMS designers have tended to bypass operating system services in favor of supplying their own. He concludes that efficient, minimal operating systems are the most desirable for supporting the kinds of database management systems that supply their own optimized services. Database support is becoming more common in today's systems. For example, Microsoft intends to use a database to store all user data in the next version of the Windows operating system.[92]

Self Review

1. Why should operating systems generally avoid direct support for database systems?
2. Why might more operating systems directly support database systems in the future?

Ans: **1)** Each database system may have different needs, which are difficult for the operating system to predict. These database systems generally exhibit higher performance when supplying their own optimized services. **2)** If an operating system uses a single database as the primary way to store user information, it should certainly provide optimized services for that database.

Web Resources

www.linux-tutorial.info/cgi-bin/display.pi795&0&0&0&3
Contains a tutorial describing file systems in Linux.

www.redhat.com/docs/manuals/linux/RHL-8.0-Manual/admin-primer/sl-storage-basics,html
Overviews file system concepts.

www.itworld.com/nl/db_mgr/
Contains articles about data management strategies.

www.winnetmag.com/articles/index.cfm?articleid=3455
Includes information about the Windows NT file system.

www.osta.org/specs/pdf/udf201.pdf
Contains the 2.01 revision of the Universal Disk Format (UDF) file system, which is commonly used to organize data on DVDs.

storageconference.org/
Contains articles and presentations from the annual cooperative IEEE-NASA Storage Conference, highlighting techniques and technologies that manage mass storage devices.

www.sqlcourse.com/
Provides an interactive tutorial, teaching the basics of SQL.

Summary

Information is stored in computers according to a data hierarchy. The lowest level of the data hierarchy is composed of bits. Bits are grouped together in bit patterns to represent all data items of interest in computer systems. The next level in the data hierarchy is fixed-length patterns of bits such as bytes, characters and words. A byte is typically 8 bits. A word is the number of bits a processor can operate on at once. Characters map bytes (or groups of bytes) to symbols such as letters, numbers, punctuation and new lines. The three most popular character sets in use today are ASCII (American Standard Code for Information Interchange), EBCDIC (Extended Binary-Coded Decimal Interchange Code) and Unicode.

A field is a group of characters. A record is a group of fields. A file is a group of related records. The highest level of the data hierarchy is a file system or database. A volume is a unit of data storage that may hold multiple files.

A file is a named collection of data that may be manipulated as a unit by operations such as open, close, create, destroy, copy, rename and list. Individual data items within a file may be manipulated by operations like read, write, update, insert and delete. File characteristics include location, accessibility, type, volatility and activity. Files can consist of one or more records.

A file system organizes files and manages access to data. File systems are responsible for file management, auxiliary storage management, file integrity mechanisms and access methods. A file system primarily is concerned with managing secondary storage space, particularly disk storage.

File systems should exhibit device independence—users should be able to refer to their files by symbolic names rather than having to use physical device names. File systems should also provide backup and recovery capabilities to prevent either accidental loss or malicious destruction of information. The file system may also provide encryption and decryption capabilities to make information useful only to its intended audience.

File systems use directories, which are files containing the names and locations of other files in the file system, to organize and quickly locate files. A directory entry stores information such as a file name, location, size, type arid accessed, modified and creation times. The simplest file system organization is the single-level (or flat) system, which stores all of its files using one directory. In single-level file systems, no two files can have the same name and the file system must perform a linear search of the directory contents to locate each file, which can lead to poor performance.

In a hierarchical file system, a root is used to indicate where on the storage device the root directory begins. The root directory points to the various directories, each of which contains an entry for each of its files. File names need be unique only within a given user directory. The name of a file is usually formed as the pathname from the root directory to the file.

Many file systems support the notion of a working directory to simplify navigation using pathnames. The working directory enables users to specify a pathname that does not begin at the root directory (i.e., a relative path). When a file system encounters a relative pathname, it forms the absolute path (i.e., the path beginning at the root) by concatenating the working directory and the relative path.

A link, which is a directory entry that references a data file or directory located in a different directory, facilitates data sharing and can make it easier for users to access files located throughout a file system's directory structure. A soft link is a directory entry containing the pathname for another file. A hard link is a directory entry that specifies the location of the file (typically a block number) on the storage device. Because a hard link specifies a physical location of a file, it references invalid data when the physical location of its corresponding file changes. Because soft links store the logical location of the file in the file system, they do not require updating when file data is moved. However, if a user moves a file to different directory or renames the file, any soft links to that file are no longer valid.

Metadata is information that protects the integrity of the file system and cannot be modified directly by users. Many file systems store in a superblock critical information that protects the integrity of the file system, such as the file system identifier and the location of the storage device's free blocks. To reduce the risk of date loss, most file systems distribute redundant copies of the superblock throughout the storage device.

The file open operation returns a file descriptor which is a non-negative integer index into the open-file table. From this point on, access to the file is directed through the file descriptor. To enable fast access to file-specific information such as permissions, the open-file table often contains file control blocks, also called file attributes, which are highly system-dependent structures that might include the file's symbolic name, location in secondary storage, access control data and so on.

The mount operation combines multiple file systems into one namespace so that they can be referenced from a single root directory. The mount command assigns a directory, called the mount point, in the native file system to the root of the mounted file system. File systems manage mounted directories with mount tables, which contain information about the location of mount points and the devices to which they point. When the native file system encounters a mount point, it uses the mount table to determine the device and type of the mounted file system. Users can create soft links to files in mounted file systems but cannot create hard links between file systems.

File organization refers to the manner in which the records of a file are arranged on secondary storage. File organization schemes include sequential, direct, indexed nonsequential and partitioned.

The problem of allocating and freeing space on secondary storage is somewhat like that experienced in primary storage allocation under variable-partition multiprogramming. Because files tend to grow or shrink over time, and because users rarely know in advance how large their files will be, contiguous storage allocation systems have generally been replaced by more dynamic noncontiguous storage allocation systems.

File systems that employ contiguous allocation place file data at contiguous addresses on the storage device. One advantage of contiguous allocation is that successive logical records typically are physically adjacent to one another. Contiguous allocation schemes exhibit the same types of external fragmentation problems inherent in memory allocation for variable-partition multiprogramming systems. Contiguous allocation may result in poor performance if files grow and

shrink over time. If a file grows beyond the size originally specified and no contiguous free blocks are available, it must be transferred to a new area of adequate size, leading to additional I/O operations.

Using a sector-based linked-list noncontiguous file allocation scheme, a directory entry points to the first sector of a file. The data portion of a sector stores the contents of the file; the pointer portion points to the file's next sector. Sectors belonging to a common file form a linked list.

When performing block allocation, the system allocates blocks of contiguous sectors (sometimes called extents). In block chaining, entries in the user directory point to the first block of each file. The blocks comprising a file each contain two portions: a data block and a pointer to the next block. When locating a record, the chain must be searched from the beginning, and if the blocks are dispersed throughout the storage device (which is normal), the search process can be slow as block-to-block seeks occur. Insertion and deletion are done by modifying the pointer in the previous block.

Large block sizes can result in a considerable amount of internal fragmentation. Small block sizes may cause file data to be spread across multiple blocks dispersed throughout the storage device, leading to poor performance as the storage device performs many seeks to access all the records of a file.

Tabular noncontiguous file allocation uses tables storing pointers to file blocks to reduce the number of lengthy seeks required to access a particular record. Directory entries indicate the first block of a file. The current block number is used as an index into the block allocation table to determine the location of the next block. If the current block is the file's last block, then its block allocation table entry is null. Because the pointers that locate file data are stored in a central location, the table can be cached so that the chain of blocks that compose a file can be traversed quickly, which improves access times. However, to locate the last record of a file, the file system might need to follow many pointers in the block allocation table, which could take significant time. When a storage device contains many blocks, the block allocation table can become large and fragmented, reducing file system performance. A popular implementation of tabular noncontiguous file allocation is Microsoft's FAT file system.

In indexed noncontiguous file allocation, each file has an index block or several index blocks. The index blocks contain a list of pointers that point to file data blocks. A file's directory entry points to its index block, which may reserve the last few entries to store pointers to more index blocks, a technique called chaining.

The primary advantage of index block chaining over simple linked-list implementations is that searching may take place in the index blocks themselves. File systems typically place index blocks near the data blocks they reference, so the data blocks can be accessed quickly after their index block is loaded. Index blocks are called inodes (i.e., index nodes) in UNIX-based operating systems.

Some systems use a free list—a linked list of blocks containing the locations of free blocks—to manage the storage device's free space. The file system typically allocates blocks from the beginning of the free list and appends freed blocks to the end of the list. This technique requires little overhead to perform free list maintenance operations, but files are allocated in noncontiguous blocks, which increases file access time.

A bitmap contains one bit for each block in memory where the rth bit corresponds to the rth block on the storage device. A primary advantage of bitmaps over free lists is that the file system can quickly determine if contiguous blocks are available at certain locations on secondary storage. A disadvantage is that the file system may need to search the entire bitmap to find a free block, resulting in substantial execution overhead.

In a two-dimensional access control matrix, the entry a_{ij} is 1 if user i is allowed access to file j; otherwise $a_{ij} = 0$. In an installation with a large number of users and a large number of files, this matrix generally would be large and sparse. A technique that requires considerably less space is to control access to various user classes. User classes can include the file owner, a specified user, group, project or public. This access control data can be stored as part of the file control block and often consumes an insignificant amount of space.

Today's operating systems generally provide many access methods, such as queued and basic access methods. Queued access methods are used when the sequence in which records are to be processed can be anticipated, such as in sequential and indexed sequential accessing. The queued methods perform anticipatory buffering and scheduling of I/O operations. The basic access methods are normally used when the sequence in which records are to be processed cannot be anticipated, particularly with direct accessing.

Memory-mapped files map file data to a process's virtual address space instead of using a file system cache. Because references to memory-mapped files occur in a process's virtual address space, the virtual memory manager can make page-replacement decisions based on each process's reference pattern.

Most file systems implement backup techniques to store redundant copies of information, and recovery techniques that enable the system to restore data after a system failure. Physi-

cal safeguards such as locks and fire alarms are the lowest level of data protection. Performing periodic backups is the most common technique used to ensure the continued availability of data. Physical backups duplicate a storage device's data at the bit level. A logical backup stores file system data and its logical structure. Thus, logical backups inspect the directory structure to determine which files need to be backed up, then write these files to a backup device in a common, often compressed, archival format. Incremental backups are logical backups that store only file system data that has changed since the previous backup.

In systems that cannot tolerate loss of data or downtime, RAID and transaction logging are appropriate. If a system failure occurs during a write operation, file data may be left in an inconsistent state. Transaction-based file systems reduce data loss using atomic transactions, which perform a group of operations in their entirety or not at all. If an error occurs that prevents a transaction from completing, it is rolled back by returning the system to the state before the transaction began.

Atomic transactions can be implemented by recording the result of each operation in a log file instead of modifying existing data. Once the transaction has completed, it is committed by recording a sentinel value in the log. To reduce the time spent reprocessing transactions in the log, most transaction-based systems maintain checkpoints that point to the last transaction that has been transferred to permanent storage. If the system crashes, it need only examine transactions after the checkpoint. Shadow paging implements atomic transactions by writing modified data to a free block instead of the original block.

Log-structured file systems (LFS), also called journaling file systems, perform all file system operations as logged transactions to ensure that they do not leave the system in an inconsistent state. In LFS, the entire disk serves as a log file. New data is written sequentially in the log file's free space. Because modified directories and metadata are always written to the end of the log, an LFS might need to read the entire log to locate a particular file, leading to poor read performance. To reduce this problem, an LFS caches locations of file system metadata and occasionally writes inode maps or superblocks that indicate the location of other metadata, enabling the operating system to locate and cache file metadata quickly when the system boots. Some file systems attempt to reduce the cost of log-structured file systems by using a log only to store metadata. This ensures file system integrity with relatively low overhead but does not ensure file integrity in the event of a system failure.

Because data is written sequentially, each LFS write requires only a single seek while there is still space on disk. When the log fills, the file system's fragmented free space from reclaiming invalid blocks can reduce read and write performance to levels lower than those in conventional file systems. To address this issue, an LFS can create contiguous free space in the log by copying valid data to a contiguous region at the end of the log.

One approach to handling nonlocal file references in a computer network is to route all such requests to a file server, i.e., a computer system dedicated to resolving intercomputer file references. This approach centralizes control of these references, but the file server could easily become a bottleneck, because all client computers send all requests to the server. A better approach is to let the separate computers communicate directly with one another.

A database is a centrally controlled collection of data stored in a standardized format; a database system involves the data, the hardware on which the data resides and the software that controls access to data (called a database management system or DBMS). Databases reduce data redundancy and prevent data from being in an inconsistent state. Redundancy is reduced by combining data from separate files. Databases also facilitate data sharing.

An important aspect of database systems is data independence; i.e., applications need not be concerned with how data is physically stored or accessed. From the system's standpoint, data independence makes it possible for the storage structure and accessing strategy to be modified in response to the installation's changing requirements, but without the need to modify functioning applications.

Database languages allow database independence by providing a standard way to access information. A database language consists of a data definition language (DDL), a data manipulation language (DML) and a query language. A DDL specifies how data are organized and related and the DML enables data to be modified. A query language is a part of the DML that allows users to create queries that search the database for data that meets certain criteria. The Structured Query Language (SQL) is currently one of the most popular database languages.

A distributed database, a database that is spread throughout the computer systems of a network, facilitates efficient data access across many sets of data that reside on different computers.

Databases are based on models that describe how data and their relationships are viewed. The relational model is a

logical structure rather than a physical one; the principles of relational database management are independent of the physical implementation of data structures. A relational database is composed of relations that indicate the various attributes of an entity. Any particular element of a relation is called a tuple (row). Each attribute (column) of the relation belongs to a single domain. The number of attributes in a relation is the degree of the relation. A projection operation forms a subset of the attributes; a join operation combines relations to produce more complex relations. The relational database model is relatively easy to implement.

Various operating system services support database management systems, namely buffer pool management, the file system, scheduling, process management, interprocess communication, consistency control and paged virtual memory. Most of these features are not specifically optimized to DBMS environments, so DBMS designers have tended to bypass operating system services in favor of supplying their own.

Key Terms

absolute path—Path beginning at the root directory.

access **control matrix**—Two-dimensional listing of all the users and their access to privileges to files in the system.

accessibility (file)—File property that places restrictions on which users can access file data.

access method—Technique a file system uses to access file data. See also queued access methods and basic access methods.

activity (file)—Percentage of a file's records accessed during a given period of time.

anticipatory buffering—Technique that allows processing and I/O operations to be overlapped by buffering more than one record at a time in main memory.

ASCII (American Standard Code for Information Interchange)—Character set, popular in personal computers and in data communication systems, that stores characters as 8-bit bytes.

atomic transaction—Group of operations that have no effect on the state of the system unless they complete in their entirety.

auxiliary storage management—Component of file systems concerned with allocating space for files on secondary storage devices.

backup—Creation of redundant copies of information.

basic access **method**—File access method in which the operating system responds immediately to user I/O demands. It is used when the sequence in which records are to be processed cannot be anticipated, particularly with direct accessing.

binary relation—Relation (in a relational database) of degree 2.

bitmap—Free space management technique that maintains one bit for each block in memory, where the *i*th bit corresponds to the /th block in memory. Bitmaps enable a file system to more easily allocate contiguous blocks but can require substantial execution time to locate a free block.

bit pattern—Lowest level of the data hierarchy. A bit pattern is a group of bits that represent virtually all data items of interest in computer systems.

block allocation—Technique that enables the file system to manage secondary storage more efficiently and reduce file traversal overhead by allocating extents (blocks of contiguous sectors) to files.

blocked record—Record that may contain several logical records for each physical record.

byte—Second-lowest level in the data hierarchy. A byte is typically 8 bits.

character—In the data hierarchy, a fixed-length pattern of bits, typically 8,16 or 32 bits.

character set—Collection of characters. Popular character sets include ASCII, EBCDIC and Unicode.

chaining—Indexed noncontiguous allocation technique that reserves the last few entries of an index block to store pointers to more index blocks, which in turn point to data blocks. Chaining enables index blocks to reference large files by storing references to its data across several blocks.

checkpoint—Marker indicating which transactions in a log have been transferred to permanent storage. The system need only reapply the transactions from the latest checkpoint to determine the state of the file system, which is faster than reapplying all transactions starting at the beginning of the log.

close (file)—Operation that prevents further reference to a file until it is reopened.

committed transaction—Transaction that has completed successfully.

copy (file)—Operation that creates another version of a file with a new name.

create (file) — Operation that builds a new file.

data definition language (DDL)—Type of language that specifies the organization of data in a database.

data manipulation language (DML)—Type of language that enables data modification.

data-dependent application—Application that relies on a particular file system's organization and access techniques.

data hierarchy—Classification that groups different numbers of bits to extract meaningful data. Bit patterns, bytes and words contain small numbers of bits that are interpreted by hardware and low-level software. Fields, records and files may contain large numbers of bits that are interpreted by operating systems and user applications.

data independence—Property of applications that do not rely on a particular file organization technique or access technique.

database—Centrally controlled collection of data that is stored in a standardized format and can be searched based on logical relations between data. Databases organize data according to content as opposed to pathname, which tends to reduce or eliminate redundant information.

database language—Language that provides for organizing, modifying and querying of structured data.

database management system (DBMS)—Software that controls database organization and operations.

database system—A particular set of data, the storage devices on which it resides and the software that controls its storage and retrieval (called a database management system or DBMS).

decryption—Technique that reverses data encryption so that data can be read in its original form.

degree—Number of attributes in a relation in a relational database.

delete (file)—Operation that removes a data item from a file.

destroy (file)—Operation that removes a file from the file system.

device independence—Property of files that can be referenced by an application using a symbolic name instead of a name indicating the device on which it resides.

direct file organization—File organization technique in which a record is directly (randomly) accessed by its physical address on a direct access storage device (DASD).

directory—File storing references to other files. Directory entries often include a file's name, type, and size.

distributed database—Database that is spread throughout the computer systems of a network.

distributed file system—File system that is spread throughout the computer systems of a network.

domain—Set of possible values for attributes in a relational database system.

EBCDIC (Extended Binary-Coded Decimal Interchange Code)—Eight-bit character set for representing data in mainframe computer systems, particularly systems developed by IBM.

encryption—Technique that transforms data to prevent it from being interpreted by unauthorized users.

execute access—Permission that enables a user to execute a file.

extent—Block of contiguous sectors.

FAT file system—An implementation of tabular noncontiguous file allocation developed by Microsoft.

field—In the data hierarchy, a group of characters (e.g., a person's name, street address or telephone number).

file—Named collection of data that may be manipulated as a unit by operations such as open, close, create, destroy, copy, rename and list. Individual data items within a file may be manipulated by operations like read, write, update, insert and delete. File characteristics include location, accessibility, type, volatility and activity. Files can consist of one or more records.

file allocation table (FAT)—Table storing pointers to file data blocks in Microsoft's FAT file system.

file attribute—See file control block.

file control block—Metadata containing information the system needs to manage a file, such as access control information.

file descriptor—Non-negative integer that indexes into an opened-file table. A process references a file descriptor instead of a pathname to access file data without incurring the overhead of a directory structure traversal.

file integrity mechanism—Mechanism that ensures that the information in a file is uncorrupted. When file integrity is assured, files contain only the information they are intended to have.

file management—Component of a file system concerned with providing the mechanisms for files to be stored, referenced, shared and secured.

file organization—Manner in which the records of a file are arranged on secondary storage (e.g., sequential, direct, indexed sequential and partitioned).

file **server**—System dedicated to provide remote processes access to its files.

file system — Component of an operating system that organizes files and manages access to data. File systems are concerned with organizing files logically (using pathnames) and physically (using metadata). They also manage their storage device's free space, enforce security policies, maintain data integrity and so on.

file system identifier—Value that uniquely identifies the file system a storage device is using.

flat directory structure—File system organization containing only one directory.

format a storage device—To prepare a device for a file system by performing operations such as inspecting its contents and writing storage management metadata.

free list—Linked list of blocks that contain the addresses of free blocks.

group (file access control) —Set of users with the same file access rights (e.g., members of a group that is working on a particular project).

hard link—Directory entry specifying the location of a file on its storage device.

hierarchically structured file system—File system organization in which each directory can contain multiple subdirectories but exactly one parent.

incremental backup—Backup technique that copies only data in file system that has changed since the last backup.

index block—Block that contains a list of pointers to file data blocks.

indexed sequential file organization—File organization that arranges records in a logical sequence according to a key contained in each record.

indirect block—Index block containing pointers to data blocks in inode-based file systems.

inode—Index block in a UNIX-based system that contains the file control block and pointers to singly, doubly and triply indirect blocks of pointers to file data.

inode map—Block of metadata written to the log of a log-structured file system that indicates the location of the file system's inodes. Inode maps improve LFS performance by reducing the time required to determine file locations in the LFS.

insert (file) —Operation that adds a new data item to a file.

join (database) —Operation that combines relations.

journaling file system—See log-structured file system (LFS).

link—Directory entry that references an existing file. Hard links reference the location of the file on its storage device; soft links store the file's pathname.

list (file) —Operation that prints or displays a file's contents.

location (file)—Address of a file on a storage device or in the system's logical file organization.

log-structured file system (LFS)—File system that performs all file operations as transactions to ensure that file system data and metadata is always in a consistent state. An LFS generally exhibits good write performance because data is also appended to the end of a systemwide log file. To improve read performance, an LFS typically distributes metadata throughout the log and employs large caches to store that metadata so that the locations of file data can be found quickly.

logical backup—Backup technique that stores file data and the file system's directory structure, often in a common, compressed format.

logical block—See logical record.

logical record—Collection of data treated as a unit by software.

logical view—View of files that hides the devices that store them, their format and the system's physical access techniques.

member—Sequential subfile of a partitioned file.

memory-mapped file—File whose data is mapped to a process's virtual address space, enabling a process to reference file data as it would other data. Memory-mapped files are useful for programs that frequently access file data.

metadata—Data that a file system uses to manage files and that is inaccessible to users directly. Inodes and superblocks are examples of metadata.

mount operation—Operation that combines disparate file systerns into a single namespace so they can be accessed by a single root directory.

mount point—User-specified directory within the native file system hierarchy where the mount command places the root of a mounted file system.

mount tables—Tables that store the locations of mount points and their corresponding devices.

MS-DOS—Popular operating system for the first IBM Personal Computer and compatible microcomputers.

n-ary relation—Relation of degree *n*.

namespace—Set of files that can be identified by a file system.

Network File System (NFS)—File system implemented by Sun Microsystems in which each computer maintains a virtual file system that can act as a server and/or a client.

owner (file access control)—User who created the file.

open (file) —Operation that prepares a file to be referenced.

parent directory—In hierarchically structured file systems, the directory that points to the current directory.

partitioned file—File composed of sequential subfiles.

pathname—String identifying a file or directory by its logical name, separating directories using a delimiter (e.g.,"/" or "\"). An absolute pathname specifies the location of a file or directory starting at the root directory; a relative pathname specifies the location of a file or directory beginning at the current working directory.

physical backup—Copy of each bit of the storage device; no attempt is made to interpret the contents of its file system.

physical block—See physical record.

physical device name—Name given to a file that is specific to a particular device.

physical record—Unit of information actually read from or written to disk.

physical view—View of file data concerned with the particular devices on which data is stored, the form the data takes on those devices, and the physical means of transferring data to and from those devices.

primary key—In a relational database, a combination of attributes whose value uniquely identifies a tuple.

projection (database)—Operation that creates a subset of attributes.

public (file access control)—File that may be accessed by any member of the system's user community.

queued access method—File access method that does not immediately service user I/O demands. This can improve performance when the sequence in which records are to **be** processed can be anticipated and requests can be ordered to minimize access times.

query language—Language that allows users to search a database for data that meets certain criteria.

read access—Permission to access a file for reading.

record—In the data hierarchy, a group of fields (e.g., to store several related fields containing information about a student or a customer).

recovery—Restoration of the system's data after a failure.

read (file) — Operation that inputs a data item from a file to a process.

relation—A set of tuples in the relational model.

relational model—A model of data proposed by Codd that is the basis for most modern database systems.

relative path—Path that specifies the location of a file relative to the current working directory.

rename (file) —Operation that changes a file's name.

roll back a transaction—To return the system to the state that existed before the transaction was processed.

root—Beginning of a file system's organizational structure.

root directory—Directory that points to the various user directories.

sequential file organization—File organization technique in which records are placed in sequential physical order. The "next" record is the one that physically follows the previous record.

shadow page—Block of data whose modified contents are written to a new block. Shadow pages are one way to implement transactions.

shadow paging—Transaction implementation that writes modified blocks to a new block. The copy of the block that is unmodified is released as free space when the transaction has been committed.

single-level directory structure—See flat directory structure.

size (file)—Amount of information stored in a file.

soft link—File that specifies the pathname corresponding to the file to which it is linked.

specified user (file access control)—Identity of an individual user (other than the owner) that may use a file.

Structured Query Language (SQL)—Database language that allows users to find data items that have certain properties, and also to create tables, specify integrity constraints, manage consistency and enforce security.

superblock—Block containing information critical to the integrity of the file system (e.g., the location of the file system's free block list or bitmap, the file system identifier and the location of the file system root).

symbolic name—Device-independent name (e.g., a pathname).

ternary relations—Relation of degree 3.

tuple—Particular element of a relation.

type (file)—Description of a file's purpose (e.g., an executable program, data or directory).

unblocked record—Record containing exactly one logical record for each physical record.

Unicode—Character set that supports international languages and is popular in Internet and multilingual applications.

update (file) —Operation that modifies an existing data item in a file.

user classes—Classification scheme that specifies individual users or groups of users that can access a file.

user directory—Directory that contains an entry for each of a user's files; each entry points to where the corresponding file is stored on its storage device.

volatility (file)—Frequency with which additions and deletions are made to a file.

volume—Unit of storage that may hold multiple files.

word—Number of bits a system's processor(s) can process at once. In the data hierarchy, words are one level above bytes.

working directory—Directory that contains files that a user can access directly.

write (file) —Operation that outputs a data item from a process to a file.

write access—Permission to access a file for writing.

Exercises

I3-I A virtual memory system has page size *p* and its corresponding file system has block size *b* and fixed-length record size *r*. Discuss the various relationships among *p, b,* and r that make sense. Explain why each of these possible relationships is reasonable.

13.2 Give a comprehensive enumeration of reasons why records may not necessarily be stored contiguously in a sequential disk file.

13.3 Suppose a distributed system has a central file server containing large numbers of sequential files for hundreds of users. Give several reasons why such a system may *not* be organized to perform compaction (dynamically or otherwise) on a regular basis.

13.4 Give several reasons why it may *not* be useful to store logically contiguous pages from a process's virtual memory space in physically contiguous areas on secondary storage.

13.5 A certain file system uses "systemwide" names; i.e., once one member of the user community uses a name, that name may not be assigned to new files. Most large file systems, however, require only that names be unique with respect to a given user—two different users may choose the same file name without conflict. Discuss the relative merits of these two schemes, considering both implementation and application issues.

13.6 Some systems implement file sharing by allowing several users to read a single copy of a file simultaneously. Others provide a copy of the shared file to each user. Discuss the relative merits of each approach.

13.7 Indexed sequential files are popular with application designers. Experience has shown, however, that direct access to indexed sequential files can be slow. Why is this so? In what circumstances is it better to access such files sequentially? In what circumstances should the application designer use direct files rather than indexed sequential files?

13.8 When a computer system failure occurs, it is important to be able to reconstruct the file system quickly and accurately. Suppose a file system allows complex arrangements of pointers and interdirectory references. What measures might the file system designer take to ensure the reliability and integrity of such a system?

13.9 Some file systems support a large number of access classes while others support only a few. Discuss the relative merits of each approach. Which is better for highly secure environments? Why?

13.10 Compare the allocation space for files on secondary storage to real storage allocation under variable-partition multiprogramming.

13.11 In what circumstances is compaction of secondary storage useful? What dangers are inherent in compaction? How can these be avoided?

13.12 What are the motivations for structuring file systems hierarchically?

13.13 One problem with indexed sequential file organization is that additions to a file may need to be placed in overflow areas. How might this affect performance? What can be done to improve performance in these circumstances?

13.14 Pathnames in a hierarchical file system can become lengthy. Given that the vast majority of file references are made to a user's own files, what convention might the file system support to minimize the need for using lengthy pathnames?

13.15 Some file systems store information in exactly the format created by the user. Others attempt to optimize by compressing the data. Describe how you would implement a compression/decompression mechanism. Such a mechanism necessarily trades execution time overhead against the reduced storage requirements for files. In what circumstances is this trade-off acceptable?

13.16 Suppose that a major development in primary storage technology made it possible to produce a storage so large, so fast and so inexpensive that all the programs and data at an installation could be stored in a single "chip." How would the design of

a file system for such a single-level storage differ from that for today's conventional hierarchical storage systems?

13.17 You have been asked to perform a security audit on a computer system. The system administrator suspects that the pointer structure in the file system has been compromised, thus allowing certain unauthorized users to access critical system information. Describe how you would attempt to determine who is responsible for the security breach and how it was possible for them to modify the pointers.

13.18 In most computer installations, only a small portion of file backups are ever used to reconstruct a file. Thus there is a trade-off to be considered in performing backups. Do the consequences of not having a file backup available when needed justify the effort of performing the backups? What factors point to the need for performing backups as opposed to not performing them? What factors affect the frequency with which backups should be performed?

13.19 Sequential access from sequential files is much faster than sequential access from indexed sequential files. Why then do many applications designers implement systems in which indexed sequential files are to be accessed sequentially?

13.20 In a university environment how might the user access classes "owner," "group," "specified user," and "public" be used to control access to files? Consider the use of the computer system for administrative as well as academic computing. Consider also its use in support of grant research as well as in academic courses.

13.21 File systems provide access to native files as well as to files from other mounted file systems. What types of files might best be stored in the native file system? in other mounted file systems? What problems are unique to the management of mounted file systems?

13.22 What type and frequency of backup would be most appropriate in each of the following systems?

a. a batch-processing payroll system that runs weekly
b. an automated teller machine (ATM) banking system
c. a hospital patient billing system
d. an airline reservation system in which customers make reservations for flights as much as one year in advance
e. a distributed program development system used by a group of 100 programmers

13.23 Most banks today use online teller transaction systems. Each transaction is immediately applied to customer accounts to keep balances correct and current. Errors are intolerable, but because of computer failures and power failures, these systems do "go down" occasionally. Describe how you would implement a backup/recovery capability to ensure that each completed transaction applies to the appropriate customer's account. Also, any transactions only partially completed at the time of a system failure must not apply.

13.24 Why is it useful to reorganize indexed sequential files periodically?

What criteria might a file system use to determine when reorganization is needed?

13.25 Distinguish between queued access methods and basic access methods.

13.26 How might redundancy of programs and data be useful in the construction of highly reliable systems? Database systems can greatly reduce the amount of redundancy involved in storing data. Does this imply that database systems may in fact be less reliable than nondatabase systems? Explain your answer.

13.27 Many of the storage schemes we discussed for placing information on disk include the extensive use of pointers. If a pointer is destroyed, either accidentally or maliciously, an entire data structure may be lost. Comment on the use of such data structures in the design of highly reliable systems. Indicate how pointer-based schemes can be made less susceptible to damage by the loss of a pointer.

13.28 Discuss the problems involved in enabling a distributed file system of homogeneous computers to treat remote file accesses the same as local file accesses (at least from the user's perspective).

13.29 Discuss the notion of transparency in distributed file systems of heterogeneous computers. How does a distributed file system resolve differences between various computer architectures and operating systems to enable all systems on a network to perform remote file accesses regardless of system differences?

13.30 Figure 13.9 shows an inode data structure. Suppose a file system that uses this structure has filled up the blocks stemming from the doubly indirect pointers. How many disk accesses will it take to write one more byte to the file? Assume that the inode and free block bitmap are both completely in memory, but there is no buffer cache. Also assume that blocks do not have to be initialized.

13.31 A file system can keep track of free blocks by using either a free block bitmap or a free block list. The system has a total of t blocks of size b, u of which are used, and each block number is stored using s bits.

a. Express size (in bits) of this device's free block bitmap using the given variables.
b. Express the size (in bits) of this system's free block list in terms of the given variables. Ignore the space consumed by pointers to free list continuation blocks.

13.32 Suppose a system uses 1KB blocks and 16-bit (2-byte) addresses. What is the largest possible file size for this file system for the following inode organizations?

a. The inode contains 12 pointers directly to data blocks.
b. The inode contains 12 pointers directly to data blocks and one indirect block. Assume the indirect data block uses all 1,024 bytes to store pointers to data blocks.
c. The inode contains 12 direct pointers to data blocks, one indirect data block, and one doubly indirect data block.

13-33 Why is it useful to store fixed-length directory entries? What limitation does this impose on file names?

Suggested Projects

13.34 Prepare a research paper on the upcoming WinFS file system and compare it to the NTFS file system. What problems in the NTFS file system does WinFS address?

13.35 Prepare a research paper on the implementation and application of distributed databases.

13.36 Research how operating systems such as Windows and Linux provide file system encryption and decryption capabilities.

Suggested Simulations

13.37 Design a file system. What would you put in each of the modes (files and directories should have different information)? What would you need for metadata? After you design it, implement a simulated file system in the language of your choice. Then implement the following functions: *create, open, close, read, write, cd* (change directory), and *Is* (to list the files in a directory). If you have more time, figure out how to save your file system to a file and be able to load it later.

Recommended Reading

File system research continues to produce file system implementations that adapt to changing user demands. Golden et al. provide a summary of many common file system functions.[93] For information regarding the NT File System, see Custer"s *Inside the Windows NT File System?*[4] Log-structured file systerns are discussed in detail by Rosenblum et al.[95] Wang et al. introduce the notion of user-level custom file systems as a technique to improve the performance of I/O-bound Internet servers.[96] The bibliography for this chapter is located on our Web site at www.deitel.com/books/os3e/Bibliography.pdf.

Works Cited

1. Searle, S., "Brief History of Character Codes in North America, Europe, and East Asia," modified March 13, 2002, <tron-web.super-nova.co.jp/characcodehist.html>.
2. "The Unicode® Standard: A Technical Introduction," March 4, 2003, <www.unicode.org/standard/principles.html>.
3. Golden, D., and M. Pechura, "The Structure of Microcomputer File Systems," *Communications of the ACM,* Vol. 29, No. 3, March 1986, pp. 222-230.
4. Linux kernel source code, version 2.6.0-test2, ext2_fs.h, lines 506-511, <lxr.linux.no/source/include/linux/ext2_fs.h?v=2.6.0-test2>.
5. Linux kernel source code, version 2.6.0-test2, iso_fs.h, lines 144-156, <lxr.linux.no/source/include/linux/iso_fs.h?v=2.6.0-test2>.
6. Linux kernel source code, version 2.6.0-test2, msdos_fs.h, line-151—162, <lxr.linux. no/source/include/linux msdos_fs.h?v=2.6.0-test2>.
7. Golden. D.. and M. Pechura, "The Structure of Microcomputer File Systems," *Communications of the ACM,* Vol. 29, No. 3. March 1986, p. 224.
8. Peterson, J. L.; J. S. Quarterman; and A. Silberschatz, "4.2BSD and 4.3BSD as Examples of the UNIX System," *ACM Comparing Surveys,* Vol. 17, No.4, December 1985, p. 395.
9. Appleton. R.. "A Non-Technical Look Inside the EXT2 File System," *Kernel Korner,* August 1997.
10. Peterson, J. L.; J. S. Quarterman; and A. Silberschatz, "4.2BSD and 4.3BSD as Examples of the UNIX System," *ACM Computing Surveys,* Vol. 17, No.4, December 1985, p. 395.

11. Peterson, J. L.; J. S. Quarterman; and A. Silberschatz, "4.2BSD and 4.3BSD as Examples of the UNIX System," *ACM Computing Surveys,* Vol. 17, No.4, December 1985, p. 395.
12. Peterson, J. L.; J. S. Quarterman; and A. Silberschatz, "4.2BSD and 4.3BSD as Examples of the UNIX System," *ACM Computing Surveys,* Vol. 17, No.4, December 1985, p. 395.
13. Linux kernel source code, version 2.6.0-test2, ext2_fs_sb.h, lines 25-55, <lxr.linux.no/source/include/linux/ext2_fs_sb.h?v=2.6.0-test2>.
14. Linux kernel source code, version 2.6.0-test2, iso_fs_sb.h, lines 7-32, <lxr.linux.no/source/include/linux/iso_fs_sb.h?v=2.6.0-test2>.
15. Linux kernel source code, version 2.6.0-test2, msdos_fs_sb.h, lines 38-63, <lxr.linux.no/source/include/linux/msdos_fs_sb.h?v=2.6.0-test2>.
16. Thompson, K., "UNIX Implementation," *UNIX Time-Sharing System: UNIX Programmer's Manual,* 7th ed., Vol. 2b, January 1979, p. 9.
17. Levy, E., and A. Silberschatz, "Distributed File Systems: Concepts and Examples," *ACM Computing Surveys,* Vol. 22, No. 4, December 1990, p. 329.
18. "Volume Mount Points," *MSDN Library,* February 2003, <msdn.microsoft.com/library/default.asp?url=/library/en-us/fileio/base/volume_mount_points.asp>.
19. Thompson, K., "UNIX Implementation," *UNIX Time-Sharing System: UNIX Programmer's Manual,* 7th ed., Vol. 2b, January 1979, p. 9.
20. Larson, P., and A. Kajla, "File Organization: Implementation of a Method Guaranteeing Retrieval in One Access," *Communications of the ACM,* Vol. 27, No. 7, July 1984, pp. 670-677.
21. Enbody, R. J., and H. C. Du, "Dynamic Hashing Schemes," *ACM Computing Surveys,* Vol. 20, No. 2, June 1988, pp. 85-113.
22. Koch, P. D. L,, "Disk File Allocation Based on the Buddy System," *ACM Transactions on Computer Systems,* Vol. 5, No. 4. November 1987, pp. 352-370.
23. McKusick, M.: W. Joy; S. Leffler; and R. Fabry, "A Fast File System for UNIX," *ACM Transactions on Computer Systems,* Vol. 2, No. 3, August 1984, pp. 183-184.
24. "Description of FAT32 File System," *Microsoft Knowledge Base,* February 21, 2002, <support.microsoft.com/default.aspx?scid=kb;[LN];Q154997>.
25. Card, R; T. Ts'o, and S. Tweedie, "Design and Implementation of the Second Extended Filesystem," September 20, 2001, <e2fsprogs.sourceforge.net/ext2intro.html>.
26. Kozierok, C, "DOS (MS-DOS, PC-DOS, etc.)," *PCGuide,* April 17,2001, <www.pcguide.com/ref/hdd/file/osDOS-c.html>.
27. Kozierok, C, "Virtual FAT (VFAT)," *PCGuide,* April 17, 2001, <www.pcguide.com/ref/hdd/file/fileVFAT-c.htmlx
28. Kozierok, C, "DOS (MS-DOS, PC-DOS, etc.)," *PCGuide,* April 17,2001,<www.pcguide.com/ref/hdd/file/osDOS-c.html>.
29. "FAT32," *MSDN Library,* <msdn.microsoft.com/library/default.asp?u rl=/library/en-us/fileio/storage_29v6. asp>.
30. Paterson, X, "A Short History of MS-DOS" June 1983. <www.patersontech.com/Dos/Byte/History.html>.
31. Rojas, R., "Encyclopedia of Computers and Computer History-DOS," April 2001, <www.patersontech.com/Dos/Encyclo.htm>.
32. Hunter, D., "Tim Patterson: The Roots of DOS," March 1983, <www.patersontech.com/Dos/Softalk/Softalk.html>.
33. Hunter, D., "Tim Patterson: The Roots of DOS," March 1983, <www.patersontech.com/Dos/Softalk/Softalk.html>.
34. Paterson, T, "An Inside Look at MS-DOS," June 1983, <www.patersontech.com/Dos/Byte/InsideDos.htm>.
35. Rojas, R., "Encyclopedia of Computers and Computer History—DOS," April 2001, <www.patersontech.com/Dos/Encyclo.htm>.
36. Rojas, R., "Encyclopedia of Computers and Computer History-DOS," April 2001, <www.patersontech.com/Dos/Encyclo.htm>.
37. Hunter, D., "Tim Patterson: The Roots of DOS," March 1983, <www.patersontech.com/Dos/Softalk/Softalk.html>.
38. Paterson, T, "An Inside Look at MS-DOS," June 1983, <www.patersontech.com/Dos/Byte/InsideDos.htm>.
39. Rojas. R., "Encyclopedia of Computers and Computer History—DOS," April 2001, <www.patersontech.com/Dos/Encyclo.htm>.
40. The Online Software Museum, "CP/M: History," <museum.sysun.com/museum/cpmhist.html>.
41. The Online Software Museum, "CP/M: History," <museum.sysun.com/museum/cpmhist.html>.
42. Rojas, R., "Encyclopedia of Computers and Computer History-DOS," April 2001, <www.patersontech.com/Dos/Encyclo.htm>.
43. Hunter, D,, "Tim Patterson: The Roots of DOS," March 1983. <www.patersontech.com/Dos/Softalk/Softalk.html>.
44. Rojas, R., "Encyclopedia of Computers and Computer History-DOS," April 2001, <www.patersontech.com/Dos/Encyclo.htm>.
45. The Online Software Museum, "CP/M: History," <museum.sysun.com/museum/cpmhist.html >.
46. Rojas, R., "Encyclopedia of Computers and Computer History-DOS," April 2001, <www.patersontech.com/Dos/Encyclo.htm>.
47. McKusick, M.; W. Joy; S. Leffler; and R. Fabry, "A Fast File System for UNIX," *ACM Transactions on Computer Systems,* Vol. 2, No. 3, August 1984, p. 183.
48. Hecht, M., and J. Gabbe, "Shadowed Management of Free Disk Pages with a Linked List," *ACM Transactions on Database Systems,* Vol. 8, No. 4, December 1983, p. 505.
49. Hecht, M., and J. Gabbe, "Shadowed Management of Free Disk Pages with a Linked List," *ACM Transactions on Database Systems,* Vol. 8, No. 4, December 1983, p. 503.

50. Betourne, C, et al., "Process Management and Resource Sharing in the Multiaccess System ESOPE," *Communications of the ACM,* Vol. 13, No. 12, December 1970, p. 730.
51. Green, P., "Multics Virtual Memory—Tutorial and Reflections," <ftp://ftp.stratus.com/pub/vos/multics/pg/mvm.html>, 1999.
52. Choy, M.; H. Leong; and M. Wong, "Disaster Recovery Techniques tor Database Systems," *Communications of the ACM,* Vol. 43, No. 11, November 2000, p. 273.
53. Chen, P.; W. Ng; S. Chandra; C. Aycock; G. Rajamani; and D. Lowell, "The Rio File Cache: Surviving Operating System Crashes," *Proceedings of the Seventh International Conference on Architectural Support for Programming Languages and Operating Systems,* 1996, p. 74.
54. Choy, M.; H. Leong; and M. Wong, "Disaster Recovery Techniques for Database Systems," *Communications of the ACM,* Vol. 43, No. 11, November 2000, p. 273.
55. Hutchinson, N; S. Manley; M. Federwisch; G. Harris; D. Hitz; S. Kleiman; and S. O'Malley, "Logical vs. Physical File System Backup," *Proceedings of the Third Symposium on Operating System Design and Implementation,* 1999, pp. 244-245.
56. Hutchinson, N; S. Manley; M. Federwisch; G. Harris; D. Hitz; S. Kleiman; and S. O'Malley, "Logical vs. Physical File System Backup," *Proceedings of the Third Symposium on Operating System Design and Implementation,* 1999, pp. 240,242-243.
57. Tanenbaum, A., and R. Renesse, "Distributed Operating Systems," *Computing Surveys,* Vol. 17, No.4, December 1985, p. 441.
58. Brown, M.; K. Rolling; and E. Taft, "The Alpine File System," *ACM Transactions on Computer Systems,* Vol. 3, No. 4, November 1985, pp. 267-270.
59. Ousterhout, J. K., and M. Rosenblum, "The Design and Implementation of a Log-Structured File System," *ACM Transactions on Computer Systems,* Vol. 10, No. 1, February 1992.
60. "What's New in File and Print Services," June 16, 2003, <www.microsoft.com/windowsserver2003/evaluation/overview/technologies/fileandprint.mspx>.
61. Johnson, M. K., "Whitepaper: Red Hat's New Journaling File System: ext3," 2001, <www.redhat.com/support/wpapers/redhat/ext3/>.
62. Ousterhout, J. K., and M. Rosenblum, "The Design and Implementation of a Log-Structured File System," *ACM Transactions on Computer Systems,* Vol. 10, No. 1, February 1992.
63. Piernas J; T Cortes; and J. M. Garcia, "DualFS: A New Journaling File System without Meta-Data Duplication," *Proceedings of the 16th International Conference on Supercomputing,* 2002, p. 137.
64. Matthews, J. N; D. Roselli; A. M. Costello; R. Y. Wang; and T E. Anderson, "Improving the Performance of Log-Structured File Systems with Adaptive Methods," *Proceedings of the Sixteenth ACM Symposium on Operating Systems Principles,* 1997, pp. 238-251.
65. Birrell, A. D., and R. M. Needham, "A Universal File Server." *IEEE Transactions on Software Engineering,* Vol. SE-6, No. 5. September 1980, pp. 450-453.
66. Mitchell, J. G., and J. Dion, "A Comparison of Two Network-Based File Servers," *Proceedings of the Eighth Symposium on Operating Systems Principles,* Vol. 15, No. 5, December 1981, pp. 45^16.
67. Christodoulakis, S., and C. Faloutsos, "Design and Performance Considerations for an Optical Disk-Based, Multimedia Object Server," *Computer,* Vol 19, No. 12, December 1986, pp. 45-56.
68. Merita, S., "Serving a LAN," *LAN Magazine,* October 1988, pp. 93-98.
69. Fridrich, M., and W. Older, "The Felix File Server," *Proceedings of the Eighth Symposium on Operating System Principles,* Vol. 15. No. 5, December 1981, pp. 37-44.
70. Wah, B. W., "File Placement on Distributed Computer Systems." *Computer,Vol* 17, No. 1, January 1984, pp. 23-32.
71. Mullender, S., and A. Tanenbaum, "A Distributed File Service Based on Optimistic Concurrency Control," *Proceedings of the 10th Symposium on Operating Systems Principles,* ACM, Vol. 19. No. 5, December 1985, pp. 51-62.
72. Morris, J. H.; M. Satyanarayanan; M. H. Conner; J. H. Howard: D. S. H. Rosenthal; and F. D. Smith, "Andrew: A Distributed Personal Computing Environment," *Communications of the ACM.* Vol. 29, No. 3, March 1986, pp. 184-201.
73. Nelson, M. N; B. B. Welch; and J. K. Ousterhout, "Caching in the Sprite Network File System," *ACM Transactions on Computer Systems,* Vol. 6, No. 1, February 1988, pp. 134-154.
74. Walsh, D.; R. Lyon; and G. Sager, "Overview of the Sun Network File System," *USENIX Winter Conference,* Dallas, 1985, pp. 117-124.
75. Sandberg, R., et al. "Design and Implementation of the Sun Network File System," *Proceedings of the USENIX 1985 Summer Conference,* June 1985, pp. 119-130.
76. Sandberg, R., *The Sun Network File System: Design, Implementation and Experience,* Mountain View, CA: Sun Microsystems. Inc., 1987.
77. Lazarus, J., *Sun 386i Overview,* Mountain View; CA, Sun Microsystems, Inc., 1988.
78. Schnaidt, P., "NFS Now," *LAN Magazine,* October 1988, pp. 62-69.
79. Shepler, S.; B. Callaghan; D. Robinson; R. Thurlow; C. Beame: M. Eisler; and D. Noveck, "Network File System (NFS) version 4 Protocol." RFC 3530, April 2003, <www.ietf.org/rfc, rfc3530.txt>.
80. Date, C. J, *An Introduction Database Systems,* Reading, MA: Addison-Wesley, 1981.
81. Silberschatz, A.; H. Korth; and S. Sudarshan, *Database System Concepts,* 4th ed., New York: McGraw-Hill, 2002, pp. 3-5.
82. Silberschatz, A.; H. Korth; and S. Sudarshan, *Database System Concepts,* 4th ed., New York: McGraw-Hill, 2002, pp. 135-182.

. 83. Chen, P., "The Entity-Relationship Model—Toward a Unified View of *Data.," ACM Transactions on Database Systems, Vol.* 1, No. 1,1976, pp. 9-36.

84. Markowitz, V. M, and A. Shoshani, "Representing Extended Entity-Relationship Structures in Relational Databases: A Modular Approach," *ACM Transactions on Database Systems,Vol.* 17, No. 3,1992. pp. 423-464.

85. Winston, A., "A Distributed Database Primer," *UNIX World,* April 1988, pp. 54-63.

86. Codd, E. E, "A Relational Model for Large Shared Data Banks," *Communications of the ACM,* June 1970.

87. Codd, E. E, "Further Normalization of the Data Base Relational Model," *Courant Computer Science Symposia,* Vol. 6, *Data Base Systems,* Englewood Cliffs, NJ.: Prentice Hall, 1972.

88. Blaha, M. R.; W. J. Premerlani; and J. E. Rumbaugh, "Relational Database Design Using an Object-Oriented Methodology," *Communications of the ACM,* Vol. 13, No. 4, April 1988, pp. 414-427.

89. Codd, E. E, "Fatal Flaws in SQL," *Datamation,* Vol. 34, No. 16, August 15,1988, pp. 45-48.

90. Relational Technology, *INGRES Overview,* Alameda, CA: Relational Technology, 1988.

91. Stonebraker, M., "Operating System Support for Database Management," *Communications of the ACM,* Vol. 24, No. 7, July 1981, pp. 412-418.

92. Ibelshauser, O., "The WinFS File System for Windows Longhorn: Faster and Smarter," *Tom's Hardware Guide,* June 17, 2003, <www.tomshardware.com/storage/20030617/>.

93. Golden, D., and M. Pechura, "The Structure of Microcomputer File Systems," *Communications of the ACM,* Vol. 29, No. 3, March 1986, pp. 222-230.

94. Custer, Helen, *Inside the Windows NT File System,* Redmond, WA: Microsoft Press, 1994.

95. Rosenblum, M., and J. K. Ousterhout, "The Design and Implementation of a Log-Structured File System," *ACM Transactions on Computer Systems (TOCS),* Vol. 10, No. 1, February 1992.

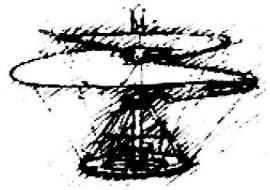
96. Wang, Jun; Rui Min; Zhuying Wu; and Yiming Hu, "Boosting I/O Performance of Internet Servers with User-Level Custom File Systems," *ACM SIGMETRICS Performance Evaluation Review,* Vol. 29, No. 2, September 2001, pp. 26-31.

Performance, Processors and Multiprocessor Management

Don't tell me how hard you work. Tell me how much you get done.
—Lewis Carroll —

Part5

The next two chapters continue our emphasis on performance. Chapter 14 discusses performance measures, monitoring, evaluation, bottlenecks, saturation and feedback loops. The chapter also examines the crucial importance of processor instruction set architectures for maximizing system performance. Chapter 15 discusses the profound performance improvement that is possible in systems that employ multiple processors, especially massive numbers of processors. In the next decade, we expect parallel computing to literally explode, with individual systems tending to become multiprocessors, and with the proliferation of distributed systems. The chapter focuses on multiprocessor systems and discusses architecture, operating system organizations, memory access architectures, memory sharing, scheduling, process migration, load balancing and mutual exclusion.



The most general definition of beauty ... Multeity in Unity.
— Samuel Taylor Coleridge —

Observe due measure, for right riming is in all things the most important factor.

—Hesiod—

Don't tell me how hard you work. Tell me how much you get done.

—James Ling—

It is not permitted to the most equitable of men to be a judge in his own cause.

—Blaise Pascal—

All words,
And no performance!

—Philip Massinger—

Chapter 14

Performance and Processor Design

Objectives

After reading this chapter, you should understand:

- *the need for performance measures.*
- *common performance metrics.*
- *several techniques for measuring relative system performance.*
- *the notions of bottlenecks, saturation and feedback.*
- *popular architectural design philosophies for processors.*
- *processor design techniques that increase performance.*

Chapter Outline

14.1 Introduction

Because an operating system is primarily a resource manager, it is important for operating systems designers, managers and users to be able to determine how effectively a particular system manages its resources. System performance measurements enable consumers to make more informed decisions and help developers build more efficient systems. In this chapter, we describe many performance issues and investigate techniques for measuring, monitoring and evaluating performance; the use of these methods can help designers, managers and users realize maximum performance from computing systems.

The performance of a system depends heavily on its hardware, operating system and the interaction between the two. Therefore, this chapter considers several evaluation techniques that measure the performance of entire systems, rather than just the operating system. We also present an introduction to popular processor-design philosophies and how each seeks to achieve high performance.

Self Review

1. How do you suppose performance evaluation benefits consumers, developers and users?
2. What resource of a computer system probably has the greatest impact on performance?

Ans: **1)** Performance evaluation provides consumers with a basis of comparison to decide between different systems, developers with useful information on how to write software to efficiently use system components and users with information that can facilitate tuning the system to meet the requirements of a specific setting. **2)** The processor(s).

14.2 Important Trends Affecting Performance Issues

In the early years of computer systems development, hardware was the dominant cost, so performance studies concentrated primarily on hardware issues. Now, hardware is relatively inexpensive and prices continue to decline. Software complexity is increasing with the widespread use of multithreading, multiprocessing, distributed systems, database management systems, graphical user interfaces and various application support systems. The software typically hides the hardware from the user, creating a virtual machine defined by the operating characteristics of the software. Cumbersome software causes poor performance, even on systems with powerful hardware, so it is important to consider a system's software performance as well as its hardware performance.

The nature of performance evaluation itself is evolving. Raw and potentially misleading measures, such as clock speed and bandwidth, have become influential in the consumer market, because vendors engineer their products with an eye to these metrics. However, other aspects of performance evaluation are improving. For example, designers have developed more sophisticated benchmarks, acquired better trace data and engineered more comprehensive computer simulation models—we investigate all these techniques later in the chapter. New industry-standard benchmarks and "synthetic programs" are emerging. Unfortunately, there is still no

consensus on these standards. Critics charge that performance results can be incorrect or misleading, because the performance evaluation techniques do not necessarily measure relevant features of a program.[1]

Self Review

1. How has the focus of performance studies shifted over the years?
2. Give several examples of how operating systems can improve performance, as discussed in the preceding chapters.

Ans: **1)** In the early years, hardware was the dominant cost, so performance studies focused on hardware issues. Today, it is recognized that sophisticated software can have a substantial effect on performance. **2)** Operating systems can improve disk and processor scheduling policies, implement more efficient thread-synchronization protocols, more effectively manage file systems, perform context switches more efficiently, improve memory management algorithms, etc.

14.3 Why Performance Monitoring and Evaluation Are Needed

In his classic paper, Lucas mentions three common purposes for performance evaluation.[2]

- **Selection evaluation**—The performance evaluator decides whether obtaining a computer system or application from a particular vendor is appropriate.
- **Performance projection**—The performance evaluator estimates the performance of a system that does not exist. It might be a completely new computer system, or an old system with a new hardware or software component.
- **Performance monitoring**—The evaluator accumulates performance data on an existing system or component to ensure that it is meeting its performance goals. Performance monitoring can also help estimate the impact of planned changes and provide system administrators with the data they need in order to make strategic decisions, such as whether to modify an existing process priority system or upgrade a hardware component.

In the early phases of a new system's development, the vendor attempts to predict the nature of applications that will run on the system and the anticipated workloads these applications must handle. Once the vendor begins development and implementation of the new system, performance evaluation and prediction are used to determine the best hardware organization, the resource management strategies that should be implemented in the operating system and whether or not the evolving system meets its performance objectives. Once the product is released to the marketplace, the vendor must be prepared to answer questions from potential users about whether the system can handle certain applications with certain levels of performance. Users are often concerned with choosing an appropriate configuration of a system that services their needs.

When the system is installed at the user's site, both the vendor and the user seek to obtain optimal performance. Administrators fine-tune the system to run at its best in the user's operating environment. This process, called **system tuning,** can often cause dramatic performance improvements, once the system is adjusted to the idiosyncrasies of the user installation.

Self Review

1. When would an evaluator use performance projection in preference to selection evaluation?
2. How does performance monitoring facilitate system tuning?

Ans: **1)** Selection evaluation help an evaluator choose among existing systems. Performance projection helps the evaluator predict the performance of systems that do not exist yet—or how anticipated modifications to existing systems would perform. **2)** An evaluator can monitor performance to determine which modifications would most likely increase system performance.

14.4 Performance Measures

By performance, we mean the efficiency with which a computer system meets its goals. Thus, performance is a relative rather than an absolute quantity, although we often talk of **absolute performance measures** such as the amount of time in which a given computer system can perform a specific computational task. However, whenever a performance measure is taken, it is normally to be used as a basis of comparison.

Performance is often "in the eye of the beholder." For example, a young music student might find a performance of Beethoven's Fifth Symphony thoroughly inspiring, whereas the conductor might be sensitive to the most minor flaws in the way a second violinist plays a certain passage. Similarly, the owner of a large airline reservation system might be pleased with the high utilization reflected by a large volume of reservations processed, whereas an individual user might be experiencing excessive delays on such a busy system.

Quantifying is difficult for some performance measures, such as **ease of use,** and simple for others, such as the speed of a disk-to-memory transfer. The performance evaluator must be careful to consider both types of measures, even though it might be possible to provide neat statistics only for the latter. Some performance measures, such as response time, are user oriented. Others, such as processor utilization, are system oriented.

Some performance results might be deceptive. For example, one operating system might focus on conserving memory by executing complicated page-replacement algorithms, whereas another might avoid these complex routines to save processor cycles for executing user programs. The former would appear more efficient on a system with a high clock speed; the latter, on a processor with a large main memory. In addition, some techniques permit the evaluator to measure the performance of small pieces of a system such as individual components or primitives. Although these tools

can be useful for pinpointing specific weaknesses, they do not tell the entire story. The evaluator might discover that an operating system performs all of its primitives efficiently except one. This should not be a major concern, unless that inefficient primitive is used extensively. Without taking into account the frequency with which each primitive is used, measurements can be misleading. Similarly, programs designed to evaluate a system for a particular environment that do not resemble the applications for which the system is intended can yield spurious results.

Some common performance measures are:

- **Turnaround time—**This is the time from submission of a job until the system returns a result to the user.
- **Response time—**This is an interactive system's turnaround time, often defined as the time from a user's pressing an *Enter* key or clicking a mouse until the system displays its response.
- **System reaction time—**In an interactive system, this is often defined as the time from a user's pressing *Enter* or clicking a mouse until the first time slice of service is given to that user's request.

These are probabilistic quantities, and in simulation and modeling studies of systems they are considered to be **random variables.** A random variable is one that can assume a certain range of values, where each value has an associated probability of occurring. We discuss the **distribution of response times,** for example, because users experience a wide range of response times on a particular interactive system over some interval of operation. A probability distribution can meaningfully characterize this range.

When we talk of the **expected value** of a random variable, we are referring to its **mean** or average value. However, means can often be deceiving. A certain mean value can be produced by averaging a series of identical or nearly identical values, or it can be produced by averaging a wide variety of values, some much larger and some much smaller than the calculated mean. Therefore, other performance measures often employed are:

- **Variance in response times** (or of any of the random variables we discuss)—The variance of response times is a measure of **dispersion.** A small variance indicates that the response times experienced by users are generally close to the mean. A large variance indicates that some users are experiencing response times that differ widely from the mean. Some users could be receiving fast service, while others could be experiencing long delays. Thus the variance of response times is a measure of **predictability;** this can be an important performance measure for users of interactive systems.
- **Throughput—**This is the work-per-unit-time performance measurement.
- **Workload—**This is the measure of the amount of work that has been submitted to the system. Often, evaluators define an acceptable level of performance for the typical workload of a computing environment. The system is evaluated compared to the acceptable level.

- **Capacity**—This is a measure of the maximum throughput a system can attain, assuming that whenever the system is ready to accept more jobs, another job is immediately available.
- **Utilization**—This is the fraction of time that a resource is in use. Utilization can be a deceptive measure. Although a high-percent utilization seems desirable, it might be the result of inefficient usage. One way to achieve high processor utilization, for example, is to run a process that is in an infinite loop! Another view of processor utilization also yields interesting insights. We might view a processor at any moment as being idle, in user mode, or in kernel mode. When a processor is in user mode, it is performing operations on behalf of a user. When the processor is in kernel mode, it is performing tasks for the operating system. Some of this time, such as context-switching time, is pure overhead. This overhead component can become large in some systems. Thus, when we measure processor utilization, we must be concerned with how much of this usage is productive work on behalf of the users, and how much is system overhead. Strangely, "poor utilization" is actually a positive measure in certain kinds of systems, such as hard real-time systems, where the system resources must stand ready to respond immediately to incoming tasks, or lives could be at risk. Such systems focus on immediate response rather than resource utilization.

Self Review

1. What is the difference between response time and system reaction time?
2. (T/F) When a processor spends most of its time in user mode, the system achieves efficient processor utilization.

Ans: **1)** Response time is the time required for the system to *finish* responding to a user request (from the time the request is submitted); system reaction time is the time required for the system to *begin* responding to a user request (from the time the request is submitted). **2)** False. If the processor is executing an infinite loop, this system is not using the processor efficiently.

14.5 Performance Evaluation Techniques

Now that we have considered some possible performance measures, we need some way of extracting them. In this section, we describe several important performance evaluation techniques.[3, 4, 5] Some of these isolate different components of a system and report each component's individual performance, allowing developers to identify areas of inefficiency. Others are oriented toward the system as a whole and allow consumers to make comparisons between systems. Still other techniques are application specific and therefore allow only indirect comparisons with other systems,. In the sections that follow, we describe various evaluation techniques and the aspects of system performance they measure.

14.5.1 Tracing and Profiling

Ideally, a system evaluator would measure the performance of several systems, each executing in the same environment. However, this is usually not feasible, especially for corporations whose environments are complex and difficult to duplicate. Such a process can be invasive, compromising the integrity of the environment and nullifying the results. When the performance of a system must be evaluated in a given environment, designers often use **trace** data. A trace is a record of system activity—typically a log of user and application requests to the operating system.

System evaluators can use trace data to characterize a particular system's execution environment by determining the frequency with which user-mode processes request particular kernel services. Before installing a new computing system, evaluators can test the new system using a workload derived from the trace data or using the trace itself. Trace data often can be modified to evaluate "what-if" scenarios. For example, a system administrator might need to determine how a new Web site will affect Web server performance. An existing trace can be modified to estimate how the system will handle its new load.[6]

When operating systems execute in similar environments, standard traces can be developed and executed on those systems to compare performance. Trace data obtained from one installation, however, might not be applicable to another. Such data is, at best, an approximation of system activity at the other installation. In addition, user logs are considered proprietary to the system on which they were recorded; rarely are such logs distributed to the research community or to vendors. Consequently, there is a dearth of trace data available for comparison and evaluation.[7]

Another method for capturing a computing system's execution environment is profiling. **Profiles** record system activity while executing in kernel mode, which can include operations such as process scheduling, memory management and I/O management. For example, a profile might record which kernel operations are performed most often. Alternatively, a profile can simply log all function calls issued by the operating system. Profiles indicate which operating system primitives are most heavily used, enabling system administrators to identify potential targets for optimization and tuning.[8] Evaluators must often employ other performance evaluation techniques in concert with profiles to determine the most effective ways to improve system performance.

Self Review

1. How do evaluators use trace data?
2. Explain the difference between traces and profiles.

Ans: **1)** Trace data permits evaluators to compare the performance of many different systems that operate in the same computing environment. Trace data describes this environment so that evaluators can obtain performance results relevant to the systems' intended use **2)** Traces record user requests, whereas profiles log all activity in kernel mode. Therefore traces describe a computing environment by capturing the user demand for particular kernel

services (without regard to the underlying system), and profiles capture operating system activity in a given environment.

14.5.2 Timings and Microbenchmarks

Timings provide a means of performing quick comparisons of computer hardware. Early computer systems were often evaluated by their add times or their memory-cycle times. Timings are useful for indicating the "raw horsepower" of a particular computer system, often in terms of the number of **MIPS (millions of instructions per second)** or **BIPS (billions of instructions per second)** it executes. Some computers perform in the **TIPS (trillion instructions per second)** range.

With the advent of **families of computers,** such as the IBM 360 series, first introduced in 1964, or the Intel Pentium series, a descendant of the Intel x86 series introduced in 1978, it has become common for hardware vendors to offer computers that enable a user to upgrade to faster processors (without replacing other computer components) as the user's needs grow. The computers in a family are compatible in that they can run the same programs but at greater speeds as the user moves up in the family. Timings provide a convenient means for comparing the members of a family of computers.

A **microbenchmark** measures the time required to perform an operating system operation (e.g., process creation). Microbenchmarks are useful for measuring how a design change affects the performance of a specific operation. **Microbenchmark suites** are programs that measure the performance of a number of important operating system primitives, such as memory operations, process creation and context-switch latency.[9] Evaluators also use microbenchmarks to measure system performance for specific operations, such as read/write bandwidth (i.e., how much data the system can transfer per unit time during a read or write) and network connection latency.[10]

Microbenchmarks describe how quickly a system performs a particular operation, not how often that operation is performed. Consequently, they do not measure important evaluation criteria such as throughput and utilization. Microbenchmarks, however, are useful in isolating which operations could be causing a system to perform poorly when coupled with information about how each operation is used.[11]

Until the 1990s, no single microbenchmark suite demonstrated the effect of hardware components on the performance of operating system primitives. In 1995 the **Imbench microbenchmark suite,** which enabled evaluators to measure and compare system performance on a variety of UNIX platforms, was introduced.[12] Although Imbench provided useful performance evaluation data that allowed comparison across multiple platforms, it was inconsistent in the way that it reported statistical data—some tests returned results based on an average of runs while other used just one run of the microbenchmark. Imbench was also limited to performing measurements once per millisecond because it used coarse software timing mechanisms; this is insufficient for measuring fast operations and timing hardware. Researchers at Harvard University addressed these limitations by creating the

hbench microbenchmark suite, which provides a standard and rigorous model for reporting statistics, enabling evaluators to more effectively analyze the relationship between operating system primitives and hardware components.[13] lmbench and hbench represent different microbenchmark philosophies, lmbench focuses on portability, which permits evaluators to compare performance across different architectures; hbench focuses on the relationship between the operating system and its underlying hardware within a particular system.[14, 15]

Self Review

1. How can results from timings and microbenchmarks be misleading? How are they useful?
2. Which performance measure can be combined with microbenchmarks to evaluate operating system performance?

Ans: **1)** Microbenchmarks measure the time required to perform specific primitives (e.g.. process creation), and timings perform quick comparisons of hardware operations (e.g., add instructions). Neither of these measurements reflects system performance as a whole. However, microbenchmarks and timings can be useful in pinpointing potential areas of inefficiency and evaluating the effect of small modifications on system performance. **2)** Profiles, which record how often an operating system primitive is used, can be combined with microbenchmarks to evaluate operating system performance.

14.5.3 Application-Specitic Evaluation

Although "raw performance" is an important measure, many users are more interested in how well particular applications will perform on a particular system. Seltzer et al. describe a **vector-based methodology** for calculating an application-specific evaluation of a system by combining trace and profile data with timings and microbenchmarks.[16]

In this technique, an evaluator records the results of microbenchmarks for the operating system's primitives. Next, the evaluator constructs a vector by placing the values corresponding to the microbenchmark results in the elements of the vector: this is called the **system vector.** Next, the evaluator profiles the operating system while executing the target application. The evaluator constructs a second vector by inserting the relative demand for each operating system primitive in an element in the vector; this is called the **application vector.** Each element in the system vector describes how long the operating system needs to execute a particular primitive, and the corresponding entry in the application vector describes the application's relative demand for that primitive. For example, if the first entry in the system vector records process creation performance, the first entry in the application vector records how many processes were created while executing a particular application (or group of applications). A characterization of the performance of a given system executing a particular application is calculated by

$$\sum_{i=1}^{n} s_i \times a_i$$

where s_i is the i^{th} entry in the system vector, a_i is the i^{th} entry in the application vector and n is the size of both vectors.[17]

This technique can be useful for comparing how efficiently different operating systems execute a particular application (or group of applications) by considering the demand an application places on each of a system's primitives. The vector-based methodology can be used to select operating system primitives to tune to improve system performance.

Some application behavior depends both on the particular application and on user input. For example, the type of application requests generated in a database system depends on its population of active users. Simply profiling the system without determining the typical stream of user requests can produce misleading results. In such cases, both the application and user requests determine the system's execution environment. Therefore, to provide a more accurate system evaluation, the vector-based methodology can be combined with a trace (Seltzer et al. call this the **hybrid methodology).** In this case, the trace data allows the evaluator to construct the application vector while accounting for how the specific application and typical stream of user requests affect the demand for each operating system primitive.[18]

A **kernel program** is another application-specific performance evaluation tool, although it is not often used. A kernel program can range from an entire program that is typical of one executed at an installation or a simple algorithm such as a matrix inversion. Using the manufacturer's estimated instruction timings, execution of a kernel program is timed for a given machine. Machines are then compared on the basis of differences in the expected execution times. Kernel programs are actually "executed on paper" rather than being run on a particular computer. They are used in selection evaluation before a consumer purchases the system being evaluated. *[Note:* "Kernel" here should not be confused with the kernel of the operating system.][19]

Kernel programs give better results than either timings or microbenchmarks, but they require manual effort to prepare and time. One key advantage of many kernel programs is that they are complete programs, which ultimately is what the user actually runs on the computer system under consideration.

Kernel programs can be helpful in evaluating certain software components of a system. For example, two different compilers might produce dramatically different code, and kernel programs can help an evaluator decide which one generates more efficient code.[20]

Self Review

1. What is a benefit of application-specific evaluation? What is a drawback?
2. Why do you suppose kernel programs are rarely used?

Ans: **1)** Application-specific evaluation is useful in determining whether a system will perform well in executing the particular programs at a given installation. A drawback is that a system must be evaluated by each installation that is considering using it; the system's designers cannot simply publish one set of performance results. **2)** Kernel programs require time and effort to prepare and time. Also, they are "executed on paper" and can introduce human

error. Often, it is easier to run the actual program, or one similar to it, on the actual system than calculate the execution time for a kernel program.

14.5.4 Analytic Model

Analytic models are mathematical representations of computer systems or their components.[21, 22, 23, 24, 25] Many types of models are used; those of queuing theory and Markov processes are two of the more popular, because they are relatively manageable and useful.

For evaluators who are mathematically inclined, the analytic model can be relatively easy to create and modify. A large body of mathematical results exists that evaluators can apply in order to estimate the performance of a given computer system or component quickly and fairly accurately. However, several disadvantages of analytic modeling hinder its applicability. One is that evaluators must be highly skilled mathematicians; these people are rare in commercial computing environments. Another is that complex systems are difficult to model precisely; as systems become more complex, analytic modeling becomes less useful.

Today's systems are often so complex that the modeler is forced to make many simplifying assumptions that can diminish the usefulness and applicability of the model. Therefore, an evaluator should use other techniques (e.g.; microbenchmarks) in concert with analytic models. Sometimes the results of an evaluation using only analytic models might be invalidated by studies using other techniques. Often the different evaluations tend to reinforce one another, demonstrating the validity of the modeler's conclusions.

Self Review

1. Explain the relative merits of complex and simple analytic models.
2. What are some benefits of using analytic modeling?

Ans: **1)** Complex analytic models are more accurate, but it might not be possible to find a mathematical solution to model a system's behavior. It is easier to represent the behavior of a system with a simpler model, but it might not accurately represent the system. **2)** There is a large body of results that evaluators can draw from when creating models; analytic models can provide accurate and quick performance results; and they can be modified relatively easily when the system changes.

14.5.5 Benchmarks

A **benchmark** is a program that is executed to evaluate a machine. Commonly, a benchmark is a **production program** which is typical of many jobs at the installation. The evaluator is thoroughly familiar with the performance of the benchmark on existing equipment, so when it is run on new equipment, the evaluator can draw meaningful conclusions.[26]-[27] Several organizations, such as **Standard Performance Evaluation Corporation** (SPEC; www.specbench.org) and **Business Application Performance Corporation (BAPCo;** www.bapco.com), have developed industry-

standard benchmarks targeted for different systems (e.g., Web servers or personal computers). Evaluators can run these benchmarks to compare similar systems from different vendors.

.One advantage of benchmarks is that many already exist, so the evaluator merely needs to choose them from known production programs or use industry-standard benchmarks. No timings are taken on individual instructions. Instead, the full program is run on the actual machine using real data, so the computer does most of the work. The chance of human error is minimal because the benchmark time is measured by the computer itself. In environments such as multiprogramming, timesharing, multiprocessing, database, data communications and real-time systems, benchmarks can be particularly valuable, because they run on the actual machine in real circumstances. The effects of such complex systems can be experienced directly instead of being estimated.

Several criteria should be considered when developing a benchmark. First, the results should be repeatable; specifically, every run of the benchmark program on a certain system should produce nearly the same result. Results do not have to be identical, and seldom are, because they can be affected by environment-specific details, such as where an item is stored on disk. Second, benchmarks should accurately reflect the types of applications that will be executed on a system. Finally, the benchmark should be widely used so that more accurate comparisons can be made between systems. A good industry-standard benchmark will have all these properties; however, the latter two often lead to conflicting design decisions. A benchmark that is specific to a certain system might not be widely used; a benchmark that is designed to test multiple systems might not yield as accurate a result for a specific system.[28]

Benchmarks are useful in evaluating hardware as well as software, even under complex operating environments. They also are particularly useful in comparing the operation of a system before and after certain changes are made. Benchmarks are not useful, however, in predicting the effects of proposed changes, unless another system exists with the changes incorporated and on which the benchmarks can be run.

Benchmarks are probably the technique most widely used by organizations and consumers when determining which equipment to purchase from several competing vendors. Their popularity as tools for this purpose led to the need for industry-standard benchmarks. SPEC was founded in 1988 to promote the development of standard, relevant benchmarks. SPEC publishes a variety of benchmarks (often called SPECmarks) that can be used to evaluate systems ranging from servers to Java Virtual Machines and publishes performance results obtained using those SPECmarks for thousands of commercial systems. SPEC ratings can be useful for making an informed decision determining which computer components to purchase. However, one must first carefully determine the focus of each SPECmark test to evaluate particular platforms. For example, SPECweb measures systems on which Web servers typically execute and should not be used to compare systems that execute in different environments.[29] If the environment of a particular Web server differs from a "typical" one, the SPEC rating might not be relevant. Further-

more, some of SPEC'S benchmarks have been criticized for narrow scope, especially by those who question the assumption that a "typical" workload can accurately approximate a real-world workload for a particular system. To combat such limitations, SPEC continually redesigns its benchmarks to improve their relevance to current systems.[30, 31]

Although SPEC produces some of the best-known benchmarks, there are a number of other popular benchmarks and benchmarking organizations. BAPCo produces several benchmarks, including the popular **SYSmark benchmark** (for desktop systems), **MobileMark** (for systems installed on mobile devices) and **WebMark** (for Internet performance).[32] Other popular benchmarks include the **Transaction Processing Performance Council (TPC) benchmarks,** which target database systems,[33] and the **Standard Application (SAP) benchmarks,** which evaluate a system's scalability.[34]

Self Review

1. How can benchmarks be used to anticipate the effect of proposed changes to a system?
2. Why are there no universally accepted "standard" benchmarks?

Ans: **1)** In general, benchmarks are effective only for determining the results after a change or for performance comparisons between systems, not for anticipating the effect of proposed changes to a system. However, if the proposed changes match the configuration of an existing system, running the benchmark on that system would be useful. **2)** Benchmarks are real programs that are run on real machines, but each machine might contain a different set of hardware that runs a different mix of programs. Therefore, benchmark designers provide "typical" application mixes that are updated regularly to better approximate particular environments.

14.5.6 Synthetic Programs

Synthetic programs (also called **synthetic benchmarks)** are similar to benchmarks, except that they focus on a specific component of the system, such as the I/O subsystem and memory subsystem. Unlike benchmarks, which are typical of real applications, evaluators construct synthetic programs for specific purposes. For example, a synthetic program might target an operating system component (e.g., the file system) or might be constructed to match the instruction frequency distribution of a large set of programs. One advantage of synthetic programs is that they can isolate specific components of a system rather than test the entire system.[35, 36, 37, 38, 39]

Synthetic programs are useful in development environments. As new features become available, synthetic programs can be used to test that they are operational. Evaluators, unfortunately, do not always have sufficient time to code and debug synthetic programs, so they often seek existing benchmark programs that match the desired characteristics of a synthetic program as closely as possible. Evaluators can use synthetic programs with benchmarks and microbenchmarks for a thorough system evaluation. These three techniques provide different levels of abstraction (the system as a whole, a component of the system or a simple primitive) that, combined, give the evaluator an understanding of the performance both of the entire system and of individual parts of the system.

Although no longer used much, the **Whetstone** and the **Dhrystone** are examples of classic synthetic programs. The Whetstone measures how well systems handle floating point calculations, and was thus helpful in evaluating scientific programs. The Dhrystone measures how effectively an architecture runs systems programs. Because the Dhrystone consumes only a small amount of memory, its effectiveness is particularly sensitive to the size of a processor's cache; if the Dhrystone's data and instruction can fit in the cache, it will execute much faster than if the processor must access main memory while it executes. In fact, because the Dhrystone fits in most of today's processor caches, in effect it measures a processor's clock speed and provides no insight into how a system manages memory. One popular synthetic program used extensively today is **WinBench 99,** which tests a system's graphics, disk and video subsytsems in a Microsoft Windows environment.[40] Other popular synthetic benchmarks include **IOStone** (which tests file systems),[41] **Hartstone** (for real-time systems)[42] and **STREAM** (for the memory subsystem).[43]

Self Review

1. Explain why synthetic programs are useful for development environments.
2. Should synthetic programs alone be used for a performance evaluation? Why?

Ans: **1)** Synthetic programs can be written fairly quickly and can test specific features for correctness. **2)** No, synthetic programs are "artificial" programs used to test specific components or to characterize a large set of programs (but not one in particular). Therefore, although producing valuable results, they do not necessarily describe how the entire system will perform when executing real programs. It is usually a good idea to use a variety of performance evaluation techniques.

14.5.7 Simulation

Simulation is a technique in which an evaluator develops a computerized model of the system being evaluated.[44, 45, 46, 47, 48] The evaluator tests the model, which presumably reflects the target system, to infer performance data about this system.

With simulation it is possible to prepare a model of a system that does not exist and then run the model to see how the system might behave in certain circumstances. Of course, the simulation must eventually be **validated** against the real system to prove that it is accurate. Simulations can highlight problems early in a system's development cycle. Computerized simulators have become especially popular in the space and transportation industries, because of the severe consequences of building systems that fail.

Simulators are generally of two types:

- **Event-driven simulators—**These are controlled by events that are made to occur in the simulator according to probability distributions.[49]
- **Script-driven simulators—**These are controlled by data carefully manipulated to reflect the anticipated environment of the simulated system; evaluators derive this data from empirical observations.

Simulation requires considerable expertise on the part of the evaluator and can consume substantial computer time. Simulators generally produce huge amounts of data that must be carefully analyzed. However, once simulators are developed, they can be reused effectively and economically.

Just as with analytic models, it is difficult to model a complex system exactly with a simulation. Several common errors cause most inaccuracies. Bugs in simulators will, of course, produce erroneous performance results. Deliberate omissions, resulting from the necessity of simplifying a simulation, can invalidate results as well. This is usually more of a problem for simple simulators. However, complex simulators suffer from the third problem—lack of detail. Complex simulators attempt to model all parts of the system, but inevitably, they do not model all details exactly. The attendant errors also can hinder a simulator's effectiveness. Therefore, to achieve the most accurate performance results, it is important to validate a simulation against the real system.[50]

Self Review

1. Which produces more consistent results, event-driven or script-driven simulators?
2. (T/F) Complex simulators are always more effective than simpler ones.

Ans: **1)** Script-driven simulators produce nearly the same result each run because the system always uses the same inputs. Because event-driven simulators dynamically generate input based on probabilities, the results are less consistent. **2)** False. Although complex simulators attempt to model a system more completely, they still might not model it accurately.

14.5.8 Performance Monitoring

Performance monitoring is the collection and analysis of information regarding system performance for existing systems.[51, 52, 53] It can help determine useful performance measures, such as throughput, response times and predictability. Performance monitoring can locate inefficiencies quickly and help system administrators decide how to improve system performance.

Users can monitor performance using software or hardware techniques. Software monitors might distort performance readings, because the monitors themselves consume system resources. Familiar examples of performance monitoring software are Microsoft Windows's Task Manager[54] and the Linux proc file system[55] (see Section 20.7.4, Proc File System). Hardware monitors are generally more costly, but they have little or no impact on system performance. Many of today's processors maintain several counting registers useful for performance monitoring that record events including clockticks, TLB misses and memory operations (such as a write to main memory).[56]

Monitors generally produce huge volumes of data that must be analyzed, possibly requiring extensive computer resources. However, they indicate precisely how the system is functioning, and this information can be extremely valuable. This is particularly true in development environments in which key design decisions might be based on the observed operation of the system.

Instruction execution traces, or module execution traces, can reveal which areas of a system are most frequently used. A module execution trace might show, for example, that a small subset of modules is being used a large percentage of the time. If designers concentrate their optimization efforts on these modules, they might be able to improve system performance without expending effort and resources on infrequently used portions of the system. Figure 14.1 summarizes performance evaluation techniques.

Self Review

1. Why do software performance monitors influence a system more than hardware performance monitors?
2. Why is performance monitoring important?

Ans: **1)** Software-based performance monitors must compete for system resources that would otherwise be allocated to programs that are being evaluated. This can result in inaccurate performance measurements. Hardware performance monitors operate in parallel with other system hardware, so that measurement does not affect system performance. **2)** Performance monitoring enables administrators to identify inefficiencies, using data describing how the system is functioning.

Technique	*Description*
Trace	Record of real application requests to the operating system, which identifies a system's workload.
Profile	Record of kernel activity taken during a real session. Profiles indicate the relative usage of operating system primitives.
Timing	Raw measure of hardware performance, which can be used for quick comparisons between related systems.
Microbenchmarks	Raw measure of how quickly an operating system performs an isolated operation.
Application-specific evaluation	Evaluation that determines how efficiently a system executes a particular application.
Analytic modeling	Technique in which an evaluator builds and analyzes a mathematical model of a computer system.
Benchmark	Program typical of one that will be run on the given system, used for comparisons between systems.
Synthetic program	Program that isolates the performance of a particular operating system component.
Simulation	Technique in which a computer model of the system is evaluated. The results of the simulation must be validated against the actual system once the system is built.
Performance monitoring	Ongoing evaluation of a system once it is installed, allowing administrators to assess whether it is meeting its demands and to determine which areas of its performance require improvement.

Figure 14.1 | *Summary of performance evaluation techniques.*

14.6 Bottlenecks and Saturation

Operating systems manage collections of resources that interface and interact in complex ways. Occasionally, resources become **bottlenecks,** limiting the system's overall performance, because they perform their designated tasks slowly relative to other resources. While other system resources might have excess capacity, the bottlenecks cannot pass jobs or processes to these other resources fast enough to keep them busy.[57,58,59]

A bottleneck tends to develop at a resource when the traffic there begins to approach its capacity. We say that the resource becomes **saturated—**i.e., processes competing for its attention begin to interfere with one another, because one must wait for others to complete using the resource.[60] A classic example is a virtual memory system that is thrashing (see Chapter 11, Virtual Memory Management). This occurs in paged systems when main memory becomes overcommitted and the working sets of the various active processes cannot be maintained simultaneously in main memory. An active process interferes with another process's use of memory by forcing the system to flush some of the other process's working set to secondary storage to make room for its own working set. Saturation can be dealt with by reducing the load on the system—e.g., thrashing can be eliminated by temporarily suspending less critical, noninteractive processes, if such processes are available.

How can bottlenecks be detected? Quite simply, each resource's request queue should be monitored. When a queue begins to grow quickly, then the **arrival rate** of requests at that resource must be larger than its **service rate,** so the resource has become saturated.

Isolation of bottlenecks is an important part of tuning a system. The bottlenecks can be removed by increasing the capacity of the resources, by adding more resources of that type at that point in the system, or, again, by decreasing the load on those resources. Removing a bottleneck does not always improve throughput, however, because other bottlenecks might also exist in the system. Tuning a system often involves identifying and eliminating bottlenecks until system performance reaches satisfactory levels.

Self Review

1. Why is it important to identify bottlenecks in a system?
2. Thrashing is due to the saturation of what resource? How would an operating system detect thrashing?

Ans: **1)** Identifying bottlenecks allows designers to focus on optimizing sections of a system that are degrading performance. **2)** Main memory. The operating system would notice a high page recapture rate—the pages being paged out to make room for incoming pages would themselves quickly be paged back into main memory.

14.7 Feedback Loops

A **feedback loop** is a technique in which information about the current state of the system can affect arriving requests. These requests can be rerouted if the feedback indicates that the system might have difficulty servicing them. Feedback can be negative, in which case arrival rates might decrease, or positive, in which case arrival rates might increase. Although we divide this section to examine positive and negative feedback situations separately, they do not represent two different techniques. Rather, a request rate at a particular resource might cause either negative or positive feedback (or neither).

14.7.1 Negative Feedback

In **negative feedback** situations, the arrival rate of new requests might decrease as a result of information being fed back. For example, a motorist pulling into a gas station and observing that several cars are waiting at each pump might decide to drive down the street to less crowded station.

In distributed systems, spooled outputs can often be printed by any of several equivalent print servers. If the queue behind one server is too long, the job may be placed in a less crowded queue.

Negative feedback contributes to **stability** in queuing systems. If the scheduler assigns arriving jobs indiscriminately to a busy device, for example, then the queue behind that device might grow indefinitely (even though other devices might be underutilized).

Self Review

1. Explain how negative feedback could improve read performance in Level 1 (mirrored) RAID.
2. How does negative feedback contribute to system stability?

Ans: **1)** If one of the disks in a mirrored pair contains a large queue of read requests, some of these requests could be sent to the other disk in the pair if it contains a smaller queue.
2) Negative feedback prevents one resource from being overwhelmed while other identical resources lie idle.

14.7.2 Positive Feedback

In positive feedback situations, the arrival rate of new requests might increase as a result of information being fed back. A classic example occurs in paged virtual memory multiprocessor systems. Suppose the operating system detects that a processor is underutilized. The system might inform the process scheduler to admit more processes to that processor's queue, anticipating that this would place a greater load on the processor. As more processes are admitted, the amount of memory that can be allocated to each process decreases and page faults might increase (because the working sets of all active processes might not fit in memory). Processor utilization will actually decrease as the system thrashes. A poorly

designed operating system might then decide to admit even more processes. Of course, this would cause further deterioration in processor utilization.

Operating systems designers must be cautious when designing mechanisms to respond to positive feedback to prevent such unstable situations from developing. The effects of each incremental change should be monitored to see whether it results in the anticipated improvement. If a change causes performance to deteriorate, this signals to the operating system that it might be entering an unstable range and that resource-allocation strategies should be adjusted in order to resume stable operation.

Self Review

1. In some large server systems, users communicate requests to a "front-end" server. This server accepts the user's request and sends it to a "back-end" server for processing. How could a front-end server balance request loads among a set of equivalent back-end servers using feedback loops?
2. This section describes how positive feedback can intensify thrashing. Suggest one possible solution to this problem.

Ans: **1)** Back-end servers with a long queue of requests might send negative feedback to the front-end sever, and back-end servers that are idle might send positive feedback. The front-end server can use these feedback loops to send incoming requests to underloaded instead of overloaded servers. **2)** The system can monitor the number of page recaptures and refuse to admit further processes beyond a certain threshold.

14.8 Performance Techniques in Processor Design

A system's performance depends heavily on the performance of its processors. Conceptually, a processor can be divided into its instruction set, which is the set of machine instructions it can perform, and its implementation, which is the physical hardware. An instruction set might be composed of a few basic instructions that perform only simple functions, such as loading a value from memory into a register or adding two numbers. Alternatively, instruction sets can contain an abundance of more complex instructions such as those that solve a specified polynomial equation. The penalty for providing such a large number of complex instructions is more complex hardware, which increases processor cost and might reduce performance for simpier instructions; the benefit is that these complex routines can be performed quickly.

The instruction set architecture (ISA) of a processor is an interface the describes the processor, including its instruction set, number of registers and memory size. The ISA is the hardware equivalent to an operating system API.[61] Although a particular ISA does not specify the hardware implementation, the elements of an ISA directly affect how the hardware is constructed and therefore significantly impact performance. Approaches to IS As have evolved over the years. This section investigates these approaches and evaluates how ISA design decisions affect performance.

Self Review

1. What trade-offs must be weighed when including single instructions that perform complex routines in an instruction set?
2. Why is the choice of an ISA important?

Ans: **1)** The penalty for adding these instructions is more complex hardware, which might slow the execution of other more frequently used instructions; the benefit is faster execution of these complex routines. **2)** The ISA specifies a programming interface between hardware and low-level software; therefore, it affects how easily code can be generated for the processor and how much memory that code occupies. Also, the ISA directly affects processor hardware, which influences cost and performance.

14.8.1 Complex Instruction Set Computing (CISC)

Until the mid-1980s, the clear trend was to incorporate frequently used sections of code into single machine-language instructions, with the hope of making these functions faster to execute and easier to code in assembly language. The logic was appealing, judging by the number of ISAs that reflected these greatly expanded instruction sets—an approach that has been named **complex instruction set computing (CISC),** partly echoing the popular term reduced instruction set computing (RISC), which we discuss in the next section.[62]

CISC processors originated when most systems programs were written in assembly language. An instruction set including single instructions that each performed several operations enabled assembly-language programmers to write their programs with fewer lines of code, thus improving programmer productivity. CISC continued to be attractive when high-level languages became widely used for writing operating systems (such as the use of C and C++ in UNIX source code), because special-purpose instructions were added to fit well with the needs of optimizing compilers. Optimizing compilers alter the structure of compiled code (but not the semantics) to achieve higher performance for a particular architecture. CISC instructions mirrored the complex operations of high-level languages rather than the simple operations a processor could execute in one or two clock cycles. These complex instruction sets tended to be implemented via microprogramming. Microprogramming introduces a layer of programming below a computer's machine language; this layer specifies the actual primitive operations that a processor must perform, such as fetching an instruction from memory (see Section 2.9 for a further description). In these CISC architectures, the machine-language instructions were interpreted, so that complex instructions were performed as a series of simpler microprogrammed instructions.[63]

CISC processors became popular largely in response to the decreasing cost of hardware coupled with the increasing cost of developing software with assembly language. CISC processors attempt to move most of the complexity from the software to the hardware. As a side effect, CISC processors also reduce the size of programs, saving memory and easing debugging. Another characteristic of some CISC processors is a trend toward reducing the number of general-purpose registers, to

decrease cost and increase space available to other CISC structures, such as the instruction decoder.[64-65]

One powerful technique to increase performance that developed during the CISC era was pipelining. A pipeline divides a processor's datapath (i.e., the portion of the processor that performs operations on data) into discrete stages. For every clock cycle, a maximum of one instruction can occupy each stage, allowing a processor to perform operations on several instructions simultaneously. In the early 1960s. IBM developed the first pipelined processor, the IBM 7030 (nicknamed "Stretch"). The 7030's pipeline consisted of four stages: instruction fetch, instruction decode, operand fetch and execution. While the processor was executing one instruction, it fetched the operands for the next instruction, decoded another instruction and fetched a fourth instruction. After each clock cycle began, each instruction in the pipeline would move forward one stage; this permitted the 7030 to process up to four instructions at once, which improved performance significantly.[66, 67, 68]

As processor manufacturing technology has improved, chip size and memory bandwidth have become less of a concern. Moreover, advanced compilers can easily perform many optimization techniques previously delegated to the implementation of the CISC instruction set.[69] However, CISC processors still are popular in many personal computers; they also can be found in several high-performance computers. Intel Pentium (www.intel.com/products/desktop/processors/pentium4/) and Advanced Micro Devices (AMD) Athlon (www.amd.com/us-en/Processors/ ProductInformation/0,,30_118_756,00.html) processors are CISC processors.

Sel Review

1. (T/F) The widespread use of high-level programming languages eliminated the usefulness of complex instructions.
2. What was the motivation behind the CISC processor-design philosophy?

Ans: **1)** False. Additional complex instructions were created to fit well with the needs of optimizing compilers. **2)** Early operating systems were written primarily in assembly code, so complex instructions simplified programming, because each instruction performed several operations.

14.8.2 Reduced Instruction Set Computing (RISC)

Many of the advantages of CISC processors were rendered unimportant by advances in hardware technology and compiler design. In addition, designers realized that the microcode required to interpret complex instructions slowed the execution of the simpler instructions. Various studies indicated that a large majority of programs generated by popular compilers used only a small portion of their target processors' instruction sets. For example, an IBM study observed that the 10 most frequently executed instructions of the hundreds in the System/370 architecture (Fig. 14.2) accounted for two-thirds of the instruction executions on that machine.[70] The additional instructions incorporated into CISC instruction sets to improve software development time were being used only infrequently. The simplest instrue-

tions to implement, such as register-memory transfers (loads and stores), were the most commonly used.

Another IBM study, a static analysis of assembly-language programs written for the IBM Series/1 computer, provided more evidence that large CISC instruction sets might be inefficient. The study observed that programmers tended to generate "semantically equivalent instruction sequences" when working with the rich machine language of the IBM Series/1. The authors concluded that since it is difficult to detect such sequences automatically, either instruction sets should be made sparser, or programmers should restrict their use of instructions.[71] These observations provided compelling motivation for the notion of **reduced instruction set computing (RISC).** This processor-design philosophy emphasizes that computer architects can optimize performance by concentrating then efforts on making common instructions, such as branches, loads and stores, execute efficiently.[72, 73]

RISC processors execute common instructions efficiently by including relatively few instructions, most of which are simple and can be performed quickly (i.e., in one clock cycle). This means that much of the programming complexity is moved from the hardware to the compiler, which permits RISC processor design to be simple and to optimize for a small set of instructions. Furthermore, RISC control units are implemented in hardware, which reduces execution overhead compared to microcode. RISC instructions, which occupy fixed-length words in memory, are faster and easier to decode than variable-length CISC instructions. Additionally, RISC processors attempt to reduce the number of accesses to main memory by providing many high-speed general-purpose registers in which programs can perform data manipulation.[74]

Providing only simple, uniform-length instructions permits RISC processors to make better use of pipelines than CISC processors do. Pipelined execution in CISC architectures can be slowed by the longer processing times of complex

Opcode	*Instruction*	*% of Execution*
BC	Branch Condition	20.2
L	Load	15.5
TM	Test Under Mask	6.1
ST	Store	5.9
LR	Load Register	4.7
LA	Load Address	4.0
LTR	Test Register	3.8
BCR	Branch Register	2.9
MVC	Move Characters	2.1
LH	Load Half Word	1.8

Figure 14.2 *| The 10 most frequently executed instructions on IBM's System/370 architecture.[75] (Courtesy of International Business Machines Corporation.)*

instructions; these have the effect of idling sections of the pipeline handling simpler instructions. Additionally, pipelines on CISC machines often contain many stages and require complex hardware to support instructions of various lengths. Pipelines in RISC architectures contain few stages and are relatively straightforward to implement because most instructions require one cycle.[76]

A simple and clever optimizing technique used in many RISC architectures is called **delayed branching**.[77, 78] When a conditional branch is executed, the next sequential instruction might or might not be executed next, depending on the evaluation of the condition. Delayed branching enables this next sequential instruction to enter execution anyway; then, if the branch is not taken, this next instruction can be well along, if not completely finished. RISC optimizing compilers often rearrange machine instructions so that a useful calculation (one that must be executed regardless of whether the branch is taken) is placed immediately after the branch. Because branching occurs more frequently than most people realize (as often as every fifth instruction on some popular architectures), this can yield considerable performance gains.[79] Lilja provides a thorough analysis of delayed branching and several other techniques that can greatly reduce the so-called **branch penalty** in pipelined architectures.[80]

The most significant trade-off in the RISC design philosophy is that its simple, reduced instruction set and register-rich architecture increase context-switching complexity.[81] RISC architectures must save a large number of registers to main memory during a context switch; this reduces context-switching performance compared to CISC architectures due to an increased number of accesses to main memory. Because context switching is pure overhead and occurs frequently, this can significantly impact system performance.

In addition to increased context-switch time, there are other drawbacks to the RISC design. One interesting study tested the effect of instruction complexity on performance; the study selected three increasingly complex instruction subsets of the VAX. The researchers reached several conclusions:[82]

i. Programs written in the simplest of the instruction subsets required as much as 2.5 times the memory of equivalent complex instruction set programs.

2. The cache miss ratio was considerably larger for programs written in the simplest subset.

3. The bus traffic for programs written in the simplest subset was about double that of programs written in the most complex subset.

These results and others like them indicate that RISC architectures can have negative consequences.

Floating point operations execute faster in CISC architectures. Additionally CISC processors perform better for graphics or scientific programs, which repeatedly execute complex instructions; these programs tend to perform better on CISC machines with optimized complex instructions than on comparable RISC machines.[83]

Figure 14.3 provides a summary comparison between the RISC and CISC design philosophies. Examples of RISC processors include the UltraSPARC

Category	***Characteristics of CISC Processors***	***Characteristics of RISC Processors***
Instruction length	Variable, typically 1 to 10 bytes.	Fixed, typically 4 bytes.
Instruction decode	Via microcode.	In hardware.
Number of instructions in ISA	Many (typically several hundred), including many complex instructions.	Few (typically less than one hundred).
Number of instructions per program	Few.	Many (often about 20 percent more than for CISC).
Number of general-purpose registers	Often, few (e.g., eight in the Intel Pentium 4 processor).[84]	Many (typically, 32).
Complexity	In hardware.	In the compiler.
Ability to exploit parallelism through pipelining	Limited.	Broad.
Underlying philosophy	Implement as many operations as possible.	Make the common case fast.
Examples	Pentium, Athlon.	MIPS, SPARC, G5.

Figure 14.3 | *RISC and CISC comparison.*

(www.sun.com/processors), MIPS (www.mips.com) and G5 (www.apple.com/powermac/specs.html) processors.

Self Review

1. Why do RISC processors exploit pipelines better than CISC processors?
2. Why does context switching require more overhead on RISC than on CISC processors?

Ans: **1)** RISC instructions are of fixed length and generally require one cycle to execute, so it is easy to overlap instructions such that most stages in the pipeline are performing meaningful work. The variable-length instructions of CISC processors have the effect of idling sections of the pipeline not needed for simple instructions. **2)** RISC processors contain many more registers than CISC processors, requiring a greater number of memory transfers during a context switch.

14.8.3 Post-RISC Processors

As RISC and CISC processors have evolved, many techniques that were developed to independently increase the performance of one or the other have been integrated into both architectures.[85, 86, 87] Many of these features add complexity to the instruction set or place more responsibility on the hardware to optimize performance. This convergence of architectures has blurred the line between RISC and CISC. The philosophy behind these processors is to increase performance in any

way possible, leading Severance et al. to coin the term **fast instruction set computing (FISC)**. Other commonly used names for this modern processor design philosophy are "post-RISC" and "second-generation RISC."[88]

However, some designers adhere to the traditional terminology, "RISC" and "CISC." This school of thought—that the two types of processors have not converged—argues that RISC and CISC refer specifically to the ISA and not to how the ISA is implemented (i.e., additional hardware complexity is irrelevant). Further, they argue that the primary difference is that RISC instructions are of a uniform length and generally execute in one cycle, unlike CISC instructions. As we see in this section, most of the convergence results from the growing similarity of complex hardware in RISC and CISC machines and the expansion of the number of instructions in RISC ISAs, two aspects which these designers argue are not central to ISA design philosophies.[89]

The sections that follow describe many techniques that post-RISC processors employ to improve processor performance. As we will see, these techniques cause processor hardware to be more complex, which increases cost and strays from the RISC tenet of keeping hardware simple.

Superscalar Architecture

Superscalar execution enables more than one instruction to be executed in parallel during each clock cycle. **Superscalar architectures** include multiple execution units on a single chip and, until recently, were primarily used in CISC processors to reduce the time required to decode complex instructions. Today, superscalar execution is found in almost all general-purpose processors, because the parallelism that it allows increases performance. For example, both the Pentium 4 and G5 processors are superscalar. These architectures contain complicated hardware that ensures no two instructions executing simultaneously depend on each other. For example, when a processor executes the machine-code equivalent of an `if...then...else` control structure, the instructions inside the `then` and `else` clauses cannot be executed until the processor has determined the value of the branch condition.[90, 91, 92]

Out-of-Order Execution (OOO)

Out-of-order execution (OOO) is a technique that dynamically reorders instructions at runtime to optimize performance by isolating groups of instructions that can execute simultaneously. OOO facilitates deep pipelines and superscalar design, which require the processor or compiler to detect groups of independent instructions that can be executed in parallel.[93, 94]

Such a mechanism, now common to both RISC and CISC processors, breaks with the RISC philosophy of leaving optimization to the compiler. OOO requires significant additional hardware to detect dependencies and handle exceptions When an instruction raises an exception, the processor must ensure that the program is in the state in which it would have been after the exception if the instructions had been executed in order.[95, 96]